

Smarter Regular Expression Matching

From Theory to Tools

Ilian Kohl

September 25, 2026

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Problem Statement	4
1.3	Contributions	4
1.4	Thesis Outline	5
2	Regular Expressions	6
2.1	Syntax	6
2.2	Semantics	7
2.3	Rust Representation	8
3	Derivatives and Partial Derivatives	10
3.1	Naive Matching	10
3.2	Brzozowski Derivatives	12
3.3	Antimirov Partial Derivatives	18
4	Matching versus Parsing	23
4.1	Matching Ambiguity	23
4.2	Parse Trees as Matching Evidence	24
4.3	Disambiguation	26
5	Derivative-Based Parser	28
5.1	The Two-Pass Approach	28
5.2	The Standard Derivative-Based Parser	29
5.2.1	Building the Empty-Word Tree	29
5.2.2	Reinserting Characters via Injection	30
5.2.3	Parser	32
5.2.4	Worked Example	33
5.3	The Bit-Coded Approach	34
5.4	The Bit-Coded Derivative-Based Parser	35
5.4.1	Annotated Expressions	35
5.4.2	Annotated Derivatives	35
5.4.3	Internalizing Expressions	37
5.4.4	Fusing Bit-Lists	38
5.4.5	Extracting Bits	38
5.4.6	Decoding Bits	40
5.4.7	Simplification	42
5.4.8	Parser	43

5.4.9	Worked Example	43
6	Partial-Derivative-Based Parser	46
6.1	The Frontier Approach	46
6.2	The Standard Partial-Derivative-Based Parser	47
6.3	The Bit-Coded Approach	47
6.4	The Bit-Coded Partial-Derivative-Based Parser	47
6.4.1	The Bit-Coded Partial Derivative	47
6.4.2	Comparing the Two Constructions	51
6.4.3	Parser	51
6.4.4	Worked Example	53
7	External To Internal Regex	55
7.1	Two ASTs	55
7.2	Parsing Surface Syntax	56
7.3	Translating ExtPat into Regex	57
7.4	Substring Search	58
8	Evaluation	60
8.1	Parser Agreement	60
8.2	Dataset Preparation	61
8.2.1	Synthetic Patterns	61
8.2.2	The Mamouras Regex Dataset	61
8.3	Performance Analysis	62
8.3.1	Performance on Synthetic Patterns	62
8.3.2	Evaluation on the Mamouras Regex Dataset	63
8.4	Discussion of Results	63
8.4.1	Optimization Challenges and Insights	63
9	Related Work and Conclusion	64
9.1	Related Work	64
9.2	Conclusion	64
A	Diagnostics	66
A.1	deriv_std_rec, match	66
A.2	deriv_bc, match	68
A.3	pderiv_std, match	69
A.4	pderiv_bc, match	71
A.5	deriv_std_rec, no match	72
A.6	deriv_bc, no match	73
A.7	pderiv_std, no match	74
A.8	pderiv_bc, no match	76

1 Introduction

1.1 Motivation

Regular expressions are a fundamental formalism for describing regular languages and are among the most widely used abstractions in computer science. They underpin applications including lexical analysis, input validation, log processing, search tools, and network security. Across these applications, the problem is regular expression matching. Given a regular expression and an input string, determine whether the string belongs to the language denoted by the expression. Formally, this is the *membership problem*.

For example, the expression $(ab|a)^*$ matches "aba", as it decomposes into "ab" followed by "a", but does not match "abb", since no sequence of subexpressions can produce it.

For many applications, a Boolean answer is enough. An input validator may only need to accept or reject a password or URL. Others require more. For instance, a lexer must determine how input decomposes into tokens. Capturing groups, schema validation, and information extraction require identifying which input parts correspond to parts of the expression. Recognition alone is not enough; the matcher must also recover the *structure*.

Efficiently solving these problems remains an active research area. The choice of algorithm influences performance, memory consumption, and security. Once matching becomes parsing, the way in which match structure is recovered also influences correctness and raises the question of how to handle ambiguity when multiple matches are possible.

Existing approaches include backtracking and finite-automata-based techniques, such as nondeterministic (NFA) and deterministic (DFA) automata. These approaches have different trade-offs in implementation complexity, memory consumption, and worst-case performance. Backtracking engines are flexible and widely used, but may exhibit exponential running time on constructed inputs, leading to regular expression denial-of-service (ReDoS) vulnerabilities [Cox, 2007]. Automata-based approaches provide stronger worst-case guarantees but may require preprocessing or suffer from state-space explosion.

Derivative-based algorithms provide an alternative by transforming the regular expression as input symbols are consumed. Each derivative represents the language remaining after an input symbol has been consumed. By maintaining a correspondence between an expression and its successive residuals, derivative-based methods provide an elegant foundation for regular expression matching and, with suitable extensions, structured parsing.

However, derivative-based techniques are less common in practical regular expression tooling. Their use for practical matching, parsing, and diagnostic tools motivates this thesis: "From Theory to Tools".

1.2 Problem Statement

Derivative-based approaches provide a principled foundation for regular expression matching and parsing. Brzozowski introduced regular expression derivatives as a way of transforming a regular expression after consuming an input symbol [Brzozowski, 1964]. Antimirov later introduced partial derivatives, which represent sets of residual expressions and are closely related to nondeterministic automata [Antimirov, 1996].

Building on Brzozowski’s derivatives, Sulzmann and Lu developed a derivative-based algorithm for POSIX regular expression parsing [Sulzmann and Lu, 2014]. Their work also presents a bit-coded variant that represents parse information as a compact bit sequence, which can be decoded after a successful match rather than maintaining an explicit parse tree throughout the computation. A Haskell implementation of the POSIX parser is available in Sulzmann’s *Programming Paradigms* teaching material [Sulzmann, 2026].

Sulzmann and Lu also studied partial derivatives in the context of regular expression sub-matching [Sulzmann and Lu, 2012]. While sub-matching is not the focus of this thesis, their work provides relevant background for the partial-derivative approach. The *Programming Paradigms* teaching material also presents a bit-coded partial-derivative-based parser in Haskell.

These constructions have primarily been presented in a theoretical and functional programming setting, leaving open practical questions about execution time, memory consumption, and data representation.

This thesis investigates these questions through implementations in Rust. As a systems programming language, Rust provides fine-grained control over memory allocation and data representation, allowing the derivative-based and partial-derivative-based parsers, including their bit-coded variants, to be evaluated in practice.

The implementation also provides traced versions of the parsing algorithms that record intermediate derivative computations during matching. These traces show the transformations performed by the algorithms and help explain successful and unsuccessful matches.

1.3 Contributions

The following contributions are being made:

- **Rust implementation of derivative-based parsers.** The thesis implements derivative-based and partial-derivative-based parsing constructions in Rust, including direct and bit-coded variants. The implementations are based on the constructions described in the literature and the corresponding Haskell material, resulting in four parser variants in total.
- **Empirical evaluation of the parser variants.** The four implementations are evaluated with respect to execution time and memory consumption using synthetic benchmarks and a corpus of real-world regular expressions from Snort/Suricata, SpamAssassin, and RegexLib, following the source categories investigated by Mamouras et al. [Mamouras et al., 2024]. They are also compared with Rust’s `regex` crate and Google’s RE2.

1.4 Thesis Outline

The remainder of this thesis follows the parsing constructions from their formal concepts through their Rust implementation to their evaluation. Theory and implementation are presented together.

- **Chapter 2 - Regular Expressions.** This chapter introduces the syntax and semantics of regular expressions and the Rust representation used throughout the implementation.
- **Chapter 3 - Derivatives.** This chapter introduces Brzozowski’s derivatives and Antimirov’s partial derivatives, together with their basic matching algorithms and Rust implementations.
- **Chapter 4 - Matching vs. Parsing.** This chapter distinguishes regular expression matching from parsing and introduces the disambiguation problem.
- **Chapter 5 - Derivative Parser.** This chapter presents Sulzmann and Lu’s derivative-based POSIX parser and its Rust implementation in direct and bit-coded variants.
- **Chapter 6 - Partial-Derivative Parser.** This chapter presents the partial-derivative-based parser and its Rust implementation in direct and bit-coded variants. It also discusses the differences between the two derivative-based approaches.
- **Chapter 7 - External versus Internal Syntax.** This chapter describes the parsing of regular expression strings into an external AST and their translation into the internal representation used by the parsers.
- **Chapter 8 - Evaluation.** This chapter evaluates the four parser variants with respect to execution time, memory consumption, and correctness. The evaluation uses synthetic benchmarks and a corpus of real-world regular expressions, and compares the implementations with Rust’s `regex` crate and Google’s RE2. It also examines optimization challenges and the insights gained from the evaluation, including the derivative traces produced by the implementations.
- **Chapter 9 - Conclusion.** Finally, this chapter summarizes the findings of the thesis and discusses the practical implications of the results and directions for future work.

2 Regular Expressions

2.1 Syntax

Let Σ be an alphabet. Regular expressions over Σ are defined inductively by the grammar:

$$r, s ::= x \mid \varepsilon \mid \emptyset \mid r + s \mid r \cdot s \mid r^* \quad (x \in \Sigma). \quad (2.1)$$

In the grammar, r and s range over regular expressions, while x ranges over the alphabet Σ and denotes an arbitrary symbol, such as a , b , or c . The constants ε and $\emptyset$ represent the *empty-word* and *empty-language* expressions. The operators $+$, $\cdot$, and $*$ denote alternation, concatenation, and Kleene star, respectively.

Concatenation is written implicitly, so rs denotes $r \cdot s$. Given this notation, we adopt the precedence and associativity conventions. Unless parentheses are shown explicitly, Kleene star binds most tightly, followed by concatenation, followed by alternation. For example,

$$\begin{aligned} ab^*c &= (a \cdot (b^*)) \cdot c, \\ a + bc^* &= a + (b \cdot (c^*)). \end{aligned}$$

For repeated operators we use left association, so

$$\begin{aligned} a + b + c &= (a + b) + c, \\ abc &= (ab)c. \end{aligned}$$

The grammar above is the core syntax. Most regular expression libraries also offer one-or-more, optional, and bounded repetition (r^+ , $r^?$, $r^{\{m,n\}}$), but these add no expressive power since each can be defined by the core constructs: $r^+ = r \cdot r^*$, $r^? = r + \varepsilon$, and

$$r^{\{m,n\}} = \underbrace{r \cdot \dots \cdot r}_{m \text{ times}} \cdot \underbrace{(r + \varepsilon) \cdot \dots \cdot (r + \varepsilon)}_{n-m \text{ times}}.$$

In the implementation, expressions using such operators are translated into the core constructs before they reach the parsing algorithms.

As a running example, consider the regular expression over $\Sigma = \{0, 1\}$

$$r_{01} = (0 + 1)^* \cdot 0 \cdot 1. \quad (2.2)$$

By the conventions above, its syntax tree is $((0 + 1)^* \cdot 0) \cdot 1$. This tree represents the hierarchical structure of the expression: a Kleene star applied to the alternation $0 + 1$, followed by concatenation with the literals 0 and 1 .

The syntactic structure must be given a semantic interpretation.

2.2 Semantics

A semantic interpretation assigns a language to each regular expression. A *word* over Σ is a finite sequence of symbols from Σ . The set of all words over Σ , including the empty word ε , is denoted by Σ^* . A *language* over Σ is a subset of Σ^* .

For a regular expression r , we write $L(r)$ for the language it denotes, where $L(r) \subseteq \Sigma^*$. The definition follows the structure of the syntax tree by *structural recursion*. Each case corresponds to one production of the grammar in (2.1):

$$\begin{aligned}
 L(x) &= \{x\}, \\
 L(\varepsilon) &= \{\varepsilon\}, \\
 L(\emptyset) &= \emptyset, \\
 L(r + s) &= L(r) \cup L(s), \\
 L(r \cdot s) &= \{vw \mid v \in L(r), w \in L(s)\}, \\
 L(r^*) &= \{\varepsilon\} \cup \{w_1 \cdots w_n \mid n \geq 1, w_1, \dots, w_n \in L(r)\}.
 \end{aligned} \tag{2.3}$$

Table 2.1 applies these definitions to small expressions. The literal a denotes the language containing only the word a . The expression ε denotes the language containing only the empty word, while $\emptyset$ denotes the language containing no words. The alternation $a + b$ combines the languages of a and b to give $\{a, b\}$, and concatenation ab combines words from the languages of a and b to give $\{ab\}$. The Kleene star a^* allows zero or more concatenations of words from the language of a , giving $\{\varepsilon, a, aa, aaa, \dots\}$.

Expression r	Language $L(r)$
a	$\{a\}$
ε	$\{\varepsilon\}$
$\emptyset$	$\emptyset$
$a + b$	$\{a, b\}$
ab	$\{ab\}$
a^*	$\{\varepsilon, a, aa, aaa, \dots\}$

Table 2.1: Languages denoted by basic expressions.

For example $r_{01} = (0 + 1)^* \cdot 0 \cdot 1$ from (2.2), we compute its language step by step:

$$\begin{aligned}
 L(0) &= \{0\}, \\
 L(1) &= \{1\}, \\
 L(0 + 1) &= L(0) \cup L(1) = \{0, 1\}, \\
 L((0 + 1)^*) &= \{0, 1\}^*, \\
 L(r_{01}) &= L((0 + 1)^*) \cdot L(0) \cdot L(1) = \{w \cdot 0 \cdot 1 \mid w \in \{0, 1\}^*\}.
 \end{aligned} \tag{2.4}$$

This is the set of binary words ending in 01. For example, the words 01, 001, and 1101 belong to $L(r_{01})$, whereas ε , 0, and 010 do not.

This is the question of whether a word *matches* a regular expression: w *matches* r if $w \in L(r)$. Determining this is the *membership problem*.

2.3 Rust Representation

The grammar from (2.1) is represented directly as a Rust enum with one variant for each syntactic construct, as shown in Figure 2.1:

```
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
pub enum Regex {
    Phi, Eps, Lit(char),
    Seq(Box<Regex>, Box<Regex>), // r . s
    Alt(Box<Regex>, Box<Regex>), // r + s
    Star(Box<Regex>),           // r*
}

impl Regex {
    pub fn seq(r: Regex, s: Regex) -> Regex {
        Regex::Seq(Box::new(r), Box::new(s))
    }
    pub fn alt(r: Regex, s: Regex) -> Regex {
        Regex::Alt(Box::new(r), Box::new(s))
    }
    pub fn star(r: Regex) -> Regex { Regex::Star(Box::new(r)) }
    pub fn lit(c: char) -> Regex { Regex::Lit(c) }
}
```

Figure 2.1: The `Regex` enum and constructors. Derived traits support operations needed by parsing: `Clone` for duplication, `PartialEq`/`Eq` for comparison, `Hash` for collections.

The type carries no precedence or associativity information, as these are determined by the syntax tree. Recursive variants use `Box<Regex>` because a direct definition `Seq(Regex, Regex)` would have infinite size and `Box` stores these recursive values separately, providing finite size [The Rust Project Developers, 2026a,b]. This differs from Haskell’s heap-allocated data types. Rust requires explicit indirection since the enum sizes must be known at compile time.

Constructors encapsulate the boxing and are purely structural: they perform no simplification, and `Regex::seq(Regex::Eps, r)` yields a `Seq` node, not `r`. They take arguments by value, moving subexpressions into boxes.

The running example $r_{01} = (0 + 1)^* \cdot 0 \cdot 1$ from (2.2) is represented by the syntax tree:

$$\text{Seq}(\text{Seq}(\text{Star}(\text{Alt}(0, 1)), 0), 1). \quad (2.5)$$

In Rust, this is represented as shown in Figure 2.2, with the outermost `Seq` containing `Lit('1')` and `Seq(Star(Alt(0,1)), 0)` as its right and left children.

```
Regex::seq(  
  Regex::seq(  
    Regex::star(Regex::alt(Regex::lit('0'), Regex::lit('1'))),  
    Regex::lit('0')),  
  Regex::lit('1'))
```

Figure 2.2: Rust representation of r_{01} .

3 Derivatives and Partial Derivatives

Chapter 2 introduced the syntax and semantics of regular expressions, and defined the *membership problem*: Decide whether a word w belongs to $L(r)$. This chapter introduces three algorithms for solving it, each returning a Boolean answer: match or no match.

The first is a naive recursive matcher that follows the structure of the regular-expression semantics. The other two transform the regular expression as input symbols are consumed. Brzozowski derivatives transform it into a single residual expression, while Antimirov's partial derivatives represent the possible residual expressions as a set.

3.1 Naive Matching

A direct way to decide whether $w \in L(r)$ is to interpret the regular-expression semantics as recursive matching rules, by following the structure of the expression and searching for possible decompositions of the input. Let $w[i : j]$ denote the substring from position i to j . A recursive procedure $\text{match}(w, i, j, r)$ determines whether $w[i : j] \in L(r)$:

$$\begin{aligned}
 \text{match}(w, i, j, \epsilon) &\iff i = j, \\
 \text{match}(w, i, j, x) &\iff j = i + 1 \wedge w[i] = x, \\
 \text{match}(w, i, j, r + s) &\iff \text{match}(w, i, j, r) \vee \text{match}(w, i, j, s), \\
 \text{match}(w, i, j, r \cdot s) &\iff \exists k \in [i, j]. \text{match}(w, i, k, r) \wedge \text{match}(w, k, j, s), \\
 \text{match}(w, i, j, r^*) &\iff i = j \vee \exists k \in (i, j]. \text{match}(w, i, k, r) \wedge \text{match}(w, k, j, r^*).
 \end{aligned} \tag{3.1}$$

Each rule corresponds directly to the semantics. Concatenation does not specify where to split, so the matcher must consider all possible split points k .

For matching $(a \cdot a) \cdot a$ against **aaa**, the split points $k \in \{0, 1, 2, 3\}$ are:

$$\begin{aligned}
 &\underbrace{\text{match}(w, 0, 0, a \cdot a)}_{a \cdot a \text{ against } w[0:0]=\epsilon} \wedge \underbrace{\text{match}(w, 0, 3, a)}_{a \text{ against } w[0:3]=\text{aaa}} \\
 \vee &\underbrace{\text{match}(w, 0, 1, a \cdot a)}_{a \cdot a \text{ against } w[0:1]=a} \wedge \underbrace{\text{match}(w, 1, 3, a)}_{a \text{ against } w[1:3]=aa} \\
 \vee &\underbrace{\text{match}(w, 0, 2, a \cdot a)}_{a \cdot a \text{ against } w[0:2]=aa} \wedge \underbrace{\text{match}(w, 2, 3, a)}_{a \text{ against } w[2:3]=a} \\
 \vee &\underbrace{\text{match}(w, 0, 3, a \cdot a)}_{a \cdot a \text{ against } w[0:3]=aaa} \wedge \underbrace{\text{match}(w, 3, 3, a)}_{a \text{ against } w[3:3]=\epsilon}.
 \end{aligned} \tag{3.2}$$

At $k = 2$, both operands match, triggering a recursive search on $(a \cdot a)$ against **aa** with split points $k \in \{0, 1, 2\}$, where $k = 1$ succeeds:

$$\begin{aligned}
& \underbrace{\text{match}(w, 0, 0, a)}_{a \text{ against } w[0:0]=\epsilon} \wedge \underbrace{\text{match}(w, 0, 2, a)}_{a \text{ against } w[0:2]=\text{aa}} \\
\vee & \underbrace{\text{match}(w, 0, 1, a)}_{a \text{ against } w[0:1]=\text{a}} \wedge \underbrace{\text{match}(w, 1, 2, a)}_{a \text{ against } w[1:2]=\text{a}} \\
\vee & \underbrace{\text{match}(w, 0, 2, a)}_{a \text{ against } w[0:2]=\text{aa}} \wedge \underbrace{\text{match}(w, 2, 2, a)}_{a \text{ against } w[2:2]=\epsilon}.
\end{aligned} \tag{3.3}$$

This completes the original match through the nested split points $k = 2$ and $k = 1$.

In Rust. The recursive rules translate directly into the corresponding Rust match arms, shown in Figure 3.1. Alternation tries both branches on the same input, while concatenation and star search possible split points. The star arm clones the body to construct the recursive **Star** expression. This is necessary because the recursive call needs to own the newly constructed expression. The range-based search short-circuits at the first successful split. Nested concatenations repeatedly search smaller substrings, exploring multiple paths before succeeding. This leads to exponential backtracking in the worst case.

```

Regex::Alt(r, s) => {
    match_naive_range(word, i, j, r)
    || match_naive_range(word, i, j, s)
}
Regex::Seq(r, s) => {
    (i..=j).any(|k| {
        match_naive_range(word, i, k, r)
        && match_naive_range(word, k, j, s)
    })
}
Regex::Star(r) => {
    i == j || (i + 1..=j).any(|k| {
        match_naive_range(word, i, k, r)
        && match_naive_range(word, k, j,
            &Regex::star(*r.clone()))
    })
}

```

Figure 3.1: Alternation, concatenation, and star arms of the naive matcher.

Although inefficient, this matcher is useful as a reference implementation because it follows the regular-expression semantics directly without any intermediate transformation. The cost is visible in the worked examples: nested concatenations repeatedly search smaller substrings, each exploring all possible split points before finding a match.

The derivative approach takes a different approach. Rather than searching for a decomposition of w while keeping r fixed, it transforms r as each character of w is consumed. The resulting expression describes the language that can still match the remaining suffix.

3.2 Brzowski Derivatives

Brzowski's derivative of r with respect to a symbol x , written $r \backslash x$, is the expression describing exactly what remains of r 's language after consuming the leading symbol x of a word $w \in L(r)$ [Brzowski, 1964]:

$$L(r \backslash x) = \{ w \mid xw \in L(r) \}. \quad (3.4)$$

For $w = x_1 x_2 \cdots x_n$, consume one symbol at a time by repeatedly taking derivatives:

$$r, \quad r \backslash x_1, \quad (r \backslash x_1) \backslash x_2, \quad \dots$$

Nullability. After all symbols have been consumed, the remaining input is ε . Thus, the resulting expression must accept ε for the original word to belong to $L(r)$. This is the question of whether the expression is *nullable*. Let $n(r)$ be true exactly when $\varepsilon \in L(r)$:

$$\begin{array}{llll} n(\emptyset) = \text{false} & (\mathbf{N}_\emptyset) & n(r + s) = n(r) \vee n(s) & (\mathbf{N}_+) \\ n(\varepsilon) = \text{true} & (\mathbf{N}_\varepsilon) & n(r \cdot s) = n(r) \wedge n(s) & (\mathbf{N}_{\text{seq}}) \\ n(x) = \text{false} & (\mathbf{N}_{\text{lit}}) & n(r^*) = \text{true} & (\mathbf{N}_*) \end{array} \quad (3.5)$$

The empty language $\emptyset$ is not nullable because it contains no words, including ε , while a literal cannot match the empty word. ε and r^* always match ε . An alternation is nullable when either branch is nullable, and a concatenation only when both components are.

In Rust. The definition is transcribed one `Regex` variant at a time, using the enum of Figure 2.1. The case split of (3.5) becomes the arms of a `match`, shown in Figure 3.2.

```
pub fn nullable(r: &Regex) -> bool {
    match r {
        Regex::Phi => false,
        Regex::Eps => true,
        Regex::Lit(_) => false,
        Regex::Alt(r, s) => nullable(r) || nullable(s),
        Regex::Seq(r, s) => nullable(r) && nullable(s),
        Regex::Star(_) => true,
    }
}
```

Figure 3.2: `nullable` implementation and its six cases: $\mathbf{N}_\emptyset$, $\mathbf{N}_\varepsilon$, $\mathbf{N}_{\text{lit}}$, $\mathbf{N}_+$, $\mathbf{N}_{\text{seq}}$ and $\mathbf{N}_*$.

The expression is passed by reference because `nullable` only inspects r and does not modify or consume it. The underscores in $\mathbf{N}_{\text{lit}}$ and $\mathbf{N}_*$ indicate that the corresponding operands are irrelevant. A literal is never nullable, regardless of its symbol, while a Kleene-star expression is always nullable, regardless of its body. The short-circuiting operators `||` and `&&` avoid evaluating the second operand when the result is already determined, while preserving the corresponding logical semantics.

Derivative Rules. The rules specify the expression that remains after consuming x :

$$\begin{array}{llll}
\emptyset \backslash x = \emptyset & (D_{\emptyset}) & (r + s) \backslash x = (r \backslash x) + (s \backslash x) & (D_{+}) \\
\varepsilon \backslash x = \emptyset & (D_{\varepsilon}) & r^* \backslash x = (r \backslash x) \cdot r^* & (D_{*}) \\
y \backslash x = \begin{cases} \varepsilon & x = y \\ \emptyset & x \neq y \end{cases} & (D_{\text{lit}}) & (r \cdot s) \backslash x = \begin{cases} (r \backslash x) \cdot s + s \backslash x & n(r) = \text{true} \\ (r \backslash x) \cdot s & n(r) = \text{false} \end{cases} & \begin{matrix} (D_{\text{seq}}^n) \\ (D_{\text{seq}}) \end{matrix}
\end{array} \tag{3.6}$$

The empty language and ε cannot consume x . A literal y succeeds exactly when $x = y$, leaving ε . Alternation retains both derivatives and for r^* , one occurrence of r consumes x while further occurrences remain available.

Concatenation is the only rule that depends on nullability. If r is not nullable, r must consume the leading x , leaving $r \backslash x$ as the remainder of r while s remains untouched. If r is nullable, it may either consume x giving $(r \backslash x) \cdot s$, or r matches ε and s consumes x leaving $s \backslash x$. Hence the nullable case produces the alternation $(r \backslash x) \cdot s + s \backslash x$.

In Rust. Figure 3.3 implements (3.6) similarly, one variant at a time.

```

pub fn deriv(r: &Regex, x: char) -> Regex {
    match r {
        Regex::Phi => Regex::Phi,
        Regex::Eps => Regex::Phi,
        Regex::Lit(c) => if *c == x { Regex::Eps } else { Regex::Phi },
        Regex::Alt(r1, r2) =>
            Regex::alt(deriv(r1, x), deriv(r2, x)),
        Regex::Seq(r1, r2) => {
            let dr1 = Regex::seq(deriv(r1, x), *r2.clone());
            if nullable(r1) {
                Regex::alt(dr1, deriv(r2, x))
            } else { dr1 }
        }
        Regex::Star(r1) =>
            Regex::seq(deriv(r1, x), Regex::star(*r1.clone())),
    }
}

```

Figure 3.3: `deriv` implementation with six cases.

`deriv` differs from `nullable` in that it borrows its input but returns an owned `Regex`, since a derivative is a new expression. The `if nullable(r1)` implements the case split of (3.6), with branches in order D_{seq}^n then D_{seq} . This is the only place where a derivative rule consults nullability and where the output can be larger than the input, since the nullable branch produces an alternation while the other produces a concatenation.

The cloning in `*r2.clone()` is necessary because s appears untouched in both branches of the concatenation case, and the star arm must preserve r^* 's body both derived and un-derived. This copying mechanism drives expression growth, which justifies simplification after every derivative step.

Example: Growth without simplification. Consider $(x + y)^*$ against the word $\mathbf{xx}$:
Step 1. The expression is a star, so D_* applies with body $r = x + y$:

$$\begin{aligned} (x + y)^* \backslash \mathbf{x} &= ((x + y) \backslash \mathbf{x}) \cdot (x + y)^* & D_*, r = x + y \\ &= (\varepsilon + \emptyset) \cdot (x + y)^* & D_+, D_{\text{lit}} \end{aligned} \quad (3.7)$$

Step 2. The expression is now the concatenation $(\varepsilon + \emptyset) \cdot (x + y)^*$, and deriving it consumes the second $\mathbf{x}$. The result is an alternation because D_{seq}^n applies with $r = \varepsilon + \emptyset$ and $s = (x + y)^*$, its nullable case being selected since $n(r) = n(\varepsilon + \emptyset) = \text{true}$ by N_+ and N_ε :

$$\begin{aligned} &((\varepsilon + \emptyset) \cdot (x + y)^*) \backslash \mathbf{x} \\ &= ((\varepsilon + \emptyset) \backslash \mathbf{x}) \cdot (x + y)^* + (x + y)^* \backslash \mathbf{x} & D_{\text{seq}}^n \\ &= (\emptyset + \emptyset) \cdot (x + y)^* + (x + y)^* \backslash \mathbf{x} & D_\varepsilon \\ &= (\emptyset + \emptyset) \cdot (x + y)^* + (\varepsilon + \emptyset) \cdot (x + y)^* & D_*, D_+, D_{\text{lit}} \end{aligned} \quad (3.8)$$

After both symbols are consumed, the result is checked for nullability:

$$\begin{aligned} &n((\emptyset + \emptyset) \cdot (x + y)^* + (\varepsilon + \emptyset) \cdot (x + y)^*) \\ &= n((\emptyset + \emptyset) \cdot (x + y)^*) \vee n((\varepsilon + \emptyset) \cdot (x + y)^*) & N_+ \\ &= (n(\emptyset + \emptyset) \wedge n((x + y)^*)) \vee (n(\varepsilon + \emptyset) \wedge n((x + y)^*)) & N_{\text{seq}} \\ &= (\text{false} \wedge \text{true}) \vee (\text{true} \wedge \text{true}) & N_+, N_\emptyset, N_\varepsilon, N_* \\ &= \text{true}. \end{aligned} \quad (3.9)$$

Thus $(x + y)^*$ matches $\mathbf{xx}$.

Simplification. Derivatives can grow unboundedly. Repeatedly deriving $(x + y)^*$ by the same symbol produces expressions of increasing size. This growth results from the nullable branch of D_{seq}^n , which introduces an alternation as a redundant structure that is carried into subsequent derivatives. ε and $\emptyset$ accumulate without bound:

$$\begin{array}{ll} (x + y)^* & 4 \\ \xrightarrow{x} (\varepsilon + \emptyset) \cdot (x + y)^* & 8 \\ \xrightarrow{x} (\emptyset + \emptyset) \cdot (x + y)^* + (\varepsilon + \emptyset) \cdot (x + y)^* & 17 \\ \xrightarrow{x} (\emptyset + \emptyset) \cdot (x + y)^* + ((\emptyset + \emptyset) \cdot (x + y)^* + (\varepsilon + \emptyset) \cdot (x + y)^*) & 26 \end{array} \quad (3.10)$$

To counter the growth, expressions are simplified after each derivative step, using identities that preserve the denoted language:

$$\begin{aligned} \emptyset \cdot r &= \emptyset & (S_1) \\ \varepsilon \cdot r &= r & (S_2) \\ \emptyset + r &= r & (S_3) \end{aligned} \quad (3.11)$$

S_1 eliminates dead concatenations with $\emptyset$, S_2 eliminates identity concatenations with ε , and S_3 eliminates $\emptyset$ from alternations. Each holds equally with the unit on the right.

Applying them to (3.8), after each step, returns the expression back to $(x + y)^*$, keeping the size bounded regardless of input length:

$$\begin{aligned}
& (\emptyset + \emptyset) \cdot (x + y)^* + (\varepsilon + \emptyset) \cdot (x + y)^* \\
&= \emptyset \cdot (x + y)^* + \varepsilon \cdot (x + y)^* && S_3, \text{ twice} \\
&= \emptyset + (x + y)^* && S_1, S_2 \\
&= (x + y)^* && S_3
\end{aligned} \tag{3.12}$$

In Rust. The identities are implemented recursively, applying the rules bottom-up over the expression tree. Each arm of the inner match implements an identity from (3.11).

`simplify` takes ownership of its input and returns simplified subexpressions directly, unlike `deriv`, which borrows and constructs. This model is visible where the two compose:

```
current = simplify(deriv(&current, c));
```

`deriv` borrows `current`, constructs a new expression, and `simplify` takes ownership of that result to return it without additional copying. The recursive calls ensure that all dead code is eliminated before an expression is returned, keeping size bounded at each step. Together, derive-then-simplify forms the core loop of the matcher, transforming and cleaning the expression at every input symbol.

```
pub fn simplify(r: Regex) -> Regex {
    match r {
        Regex::Seq(r, s) => {
            let r = simplify(*r);
            let s = simplify(*s);
            match (&r, &s) {
                (Regex::Phi, _) | (_, Regex::Phi) => Regex::Phi,
                (Regex::Eps, _) => s,
                (_, Regex::Eps) => r,
                _ => Regex::seq(r, s),
            }
        }
        Regex::Alt(r, s) => {
            let r = simplify(*r);
            let s = simplify(*s);
            match (&r, &s) {
                (Regex::Phi, _) => s,
                (_, Regex::Phi) => r,
                _ if r == s => r,
                _ => Regex::alt(r, s),
            }
        }
        Regex::Star(r) => match simplify(*r) {
            Regex::Eps | Regex::Phi => Regex::Eps,
            r => Regex::star(r),
        },
        other => other,
    }
}
```

Figure 3.4: Simplification rules: S_1 and S_2 in concatenation, S_3 in both branches.

Running example: $r_{01} = (0 + 1)^* \cdot 0 \cdot 1$. Recall from (2.2) that it is left-associated as $((0 + 1)^* \cdot 0) \cdot 1$, so the outer operands are $r = (0 + 1)^* \cdot 0$ and $s = 1$. We derive against $w = 101$ one symbol at a time.

1. **Derive by 1.** At the outer node, $r = (0 + 1)^* \cdot 0$ and $s = 1$. This r is not nullable:

$$n((0 + 1)^* \cdot 0) = \text{true} \wedge \text{false} = \text{false},$$

by N_{seq} , N_* and N_{lit} . Thus D_{seq} applies without a second summand:

$$r_{01} \setminus 1 = (((0 + 1)^* \cdot 0) \setminus 1) \cdot 1.$$

At the middle node, the left operand $(0 + 1)^*$ is nullable, so D_{seq}^n applies. First,

$$\begin{aligned} (0 + 1)^* \setminus 1 &= ((0 + 1) \setminus 1) \cdot (0 + 1)^* && D_* \\ &= ((0 \setminus 1) + (1 \setminus 1)) \cdot (0 + 1)^* && D_+ \\ &= (\emptyset + \varepsilon) \cdot (0 + 1)^* && D_{\text{lit}} \\ &= (0 + 1)^* && S_3, S_2. \end{aligned}$$

Hence

$$\begin{aligned} ((0 + 1)^* \cdot 0) \setminus 1 &= (0 + 1)^* \cdot 0 + (0 \setminus 1) && D_{\text{seq}}^n \\ &= (0 + 1)^* \cdot 0 + \emptyset && D_{\text{lit}} \\ &= (0 + 1)^* \cdot 0 && S_3. \end{aligned}$$

Substituting back into the outer node gives

$$r_{01} \setminus 1 = ((0 + 1)^* \cdot 0) \cdot 1 = r_{01}. \quad (3.13)$$

Thus a 1 not yet part of the final 01 is absorbed by the leading star.

2. **Derive by 0.** The expression is unchanged. Again, $(0 + 1)^* \setminus 0$ simplifies to $(0 + 1)^*$, but now the second summand at the middle node survives:

$$\begin{aligned} ((0 + 1)^* \cdot 0) \setminus 0 &= (0 + 1)^* \cdot 0 + (0 \setminus 0) && D_{\text{seq}}^n \\ &= (0 + 1)^* \cdot 0 + \varepsilon && D_{\text{lit}}. \end{aligned}$$

No simplification applies. The outer node is still non-nullable, so

$$r_{01} \setminus 1 \setminus 0 = ((0 + 1)^* \cdot 0 + \varepsilon) \cdot 1. \quad (3.14)$$

After 10, either the trailing 01 is still owed in full, represented by $(0 + 1)^* \cdot 0$, or the consumed 0 was already its first symbol, represented by ε .

3. **Derive by 1.** The outer binding has now changed. The left operand is $(0 + 1)^* \cdot 0 + \varepsilon$, which is nullable by N_+ and N_ε . So D_{seq}^n applies and contributes the second summand $1 \setminus 1 = \varepsilon$. For the first summand, D_+ applies, reusing the result from step 1:

$$\begin{aligned} ((0 + 1)^* \cdot 0 + \varepsilon) \setminus 1 &= ((0 + 1)^* \cdot 0) \setminus 1 + (\varepsilon \setminus 1) && D_+ \\ &= (0 + 1)^* \cdot 0 + \emptyset && \text{step 1; } D_\varepsilon \\ &= (0 + 1)^* \cdot 0 && S_3. \end{aligned}$$

Therefore

$$r_{01} \setminus 1 \setminus 0 \setminus 1 = ((0 + 1)^* \cdot 0) \cdot 1 + \varepsilon = r_{01} + \varepsilon. \quad (3.15)$$

The final expression is nullable by $\mathbf{N}_+$ and $\mathbf{N}_\varepsilon$, so $101 \in L(r_{01})$. The ε summand witnesses that the match is complete. For comparison, $w = 10$ leaves $((0 + 1)^* \cdot 0 + \varepsilon) \cdot 1$. Neither branch is nullable: $(0 + 1)^* \cdot 0$ still requires a 0, while $\varepsilon \cdot 1$ still requires a 1. Hence $10 \notin L(r_{01})$.

The matcher. The implementation can be summarized by the sequence of residual expressions produced while consuming the input. For $w = x_1x_2 \cdots x_n$, the initial residual is $r_0 = r$, and each subsequent residual is obtained by deriving with respect to the next symbol and simplifying the result:

$$r_0 = r, \quad r_i = \text{simplify}(r_{i-1} \setminus x_i) \quad (1 \leq i \leq n). \quad (3.16)$$

The residual r_i denotes exactly the set of suffixes that complete the consumed prefix $x_1 \cdots x_i$ to a word in $L(r)$. Since simplification preserves the represented language, each r_i is equivalent to the unsimplified derivative after consuming that prefix. After the final symbol has been consumed, the matcher accepts when the remaining residual is nullable:

$$w \in L(r) \iff n(r_n) = \text{true}.$$

So $r_0 \xrightarrow{x_1} r_1 \xrightarrow{x_2} \cdots \xrightarrow{x_n} r_n$ can be viewed as a path through a lazily constructed DFA. Each residual expression represents a state, and $\xrightarrow{x}$ denotes the transition obtained by taking the derivative with respect to x . Unlike an explicitly constructed DFA, only the states reached by the input are constructed. A residual is an accepting state exactly when it is nullable, so acceptance is determined by whether the final residual is nullable.

In Rust. Figure 3.5 implements (3.16) by composing `deriv`, `simplify`, and `nullable`. The implementation follows the mathematical definition directly. The current residual is transformed at each step, and acceptance is determined after the complete input has been consumed.

```
pub fn match_deriv(input: &str, r: &Regex) -> bool {
    let mut current = r.clone();
    for c in input.chars() {
        current = simplify(deriv(&current, c));
    }
    nullable(&current)
}
```

Figure 3.5: `match_deriv`: the core matcher loop. For each input character, derive the current residual expression and simplify it in place. Accept if the final residual is nullable.

The loop implements the residual sequence. At each iteration, `current` contains the current residual, which is then derived and simplified before processing the next input symbol. Only the current residual is retained, since the previous residual is dropped when `current` is reassigned. In contrast to parsing, where intermediate results may be required to reconstruct a parse tree, Boolean matching requires only the final residual.

The derivative-based matcher avoids the exponential backtracking of naive matching by transforming the expression rather than searching for decompositions. Simplification keeps the resulting expressions compact, preventing the growth shown in (3.10).

3.3 Antimirov Partial Derivatives

The derivative of Section 3.2 combines alternative residuals into a single expression using $+$. Antimirov’s partial derivatives instead keep them as a *set* of expressions, one per strand of nondeterminism [Antimirov, 1996]. Writing $pd(r, x)$ for this set:

$$\begin{aligned}
 pd(\emptyset, x) &= \{\} & (P_{\emptyset}) \\
 pd(\varepsilon, x) &= \{\} & (P_{\varepsilon}) \\
 pd(y, x) &= \begin{cases} \{\varepsilon\} & x = y \\ \{\} & x \neq y \end{cases} & (P_{\text{lit}}) \\
 pd(r + s, x) &= pd(r, x) \cup pd(s, x) & (P_{+}) \\
 pd(r^*, x) &= \{ \text{smartC}(r', r^*) \mid r' \in pd(r, x) \} & (P_{*}) \\
 pd(r \cdot s, x) &= \begin{cases} \{ \text{smartC}(r', s) \mid r' \in pd(r, x) \} \cup pd(s, x) & n(r) = \text{true} \\ \{ \text{smartC}(r', s) \mid r' \in pd(r, x) \} & n(r) = \text{false} \end{cases} & \begin{matrix} (P_{\text{seq}}^n) \\ (P_{\text{seq}}) \end{matrix}
 \end{aligned} \tag{3.17}$$

The empty language and ε produce the empty set, since neither has a residual with respect to x : where $D_{\emptyset}$ and D_{ε} returned $\emptyset$ as a node in the result, $P_{\emptyset}$ and P_{ε} return no residual at all. A literal y gives $\{\varepsilon\}$ when $x = y$ and $\{\}$ otherwise. Alternation unions the two residual sets, and for r^* one iteration of r consumes x , with each resulting residual placed in front of r^* so the star may continue. None of these alternatives are combined with $+$; each stays a separate element of the set.

Concatenation depends on nullability for the same reason as D_{seq} . If r is not nullable it must consume x , and each of its residuals is concatenated with the untouched s . If r is nullable, r may instead match ε and let s consume x , in which case $pd(s, x)$ joins the set. Where D_{seq}^n combined the two cases with $+$ inside one expression, P_{seq}^n keeps them as two elements of one set.

In Rust. Figure 3.6 presents the implementation of (3.17), with each `match` arm corresponding to one of its cases. The `Phi`, `Eps`, and `Lit` variants implement $P_{\emptyset}$, P_{ε} , and P_{lit} directly, returning `Vec::new()` where the equation gives $\{\}$. `Alt` concatenates the two child vectors, implementing the union of P_{+} . `Star` maps `smart_seq` over r ’s own residuals, matching P_{*} ; `Regex::star(*r.clone())` clones because `Regex::star` needs an owned `Regex` and `r` is only borrowed, once per residual since the closure runs once for each.

`Seq` implements the case split directly. The `map` applies `smart_seq` to each residual of the left operand, matching P_{seq} , and the guarded `out.extend(pderiv(s, x))` adds $pd(s, x)$ for P_{seq}^n . Although the mathematical definition uses sets, the implementation uses `Vec<Regex>` throughout the recursion and defers deduplication to a single `HashSet` insertion at the top, rather than deduplicating at every `Alt` or `Seq` node on the way down.

Every union in `pderiv` is therefore a vector `extend`, not a set operation. Duplicate residuals reached by different strands are only merged once inserted into the `HashSet` used by the matcher below to keep the residual generation separate from deduplication.

```

fn pderiv(r: &Regex, x: char) -> Vec<Regex> {
  match r {
    Regex::Phi => Vec::new(),
    Regex::Eps => Vec::new(),
    Regex::Lit(c) => {
      if *c == x { vec![Regex::Eps] } else { Vec::new() }
    }
    Regex::Alt(r, s) => {
      let mut out = pderiv(r, x);
      out.extend(pderiv(s, x));
      out
    }
    Regex::Seq(r, s) => {
      let mut out: Vec<Regex> = pderiv(r, x)
        .into_iter()
        .map(|r_prime| smart_seq(r_prime, s))
        .collect();
      if nullable(r) {
        out.extend(pderiv(s, x));
      }
      out
    }
    Regex::Star(r) => {
      pderiv(r, x)
        .into_iter()
        .map(|r_prime| smart_seq(r_prime, &Regex::star(*r.clone())))
        .collect()
    }
  }
}

```

Figure 3.6: $pd(r, x)$ in full, corresponding arm for arm to Equation (3.17).

The smart constructor. The concatenation in P_{seq} is constructed using the auxiliary constructor $smartC$, denoted by SC:

$$smartC(r', s) = \begin{cases} s & r' = \varepsilon \\ r' \cdot s & \text{otherwise} \end{cases} \quad (3.18)$$

If $r' = \varepsilon$, the consumed symbol completed the left operand and there is nothing left to concatenate with s , so the residual is s alone. Otherwise r' stays in front of s . $smartC$ therefore states the concatenation rule without ever constructing $\varepsilon \cdot s$.

A set element is one possible residual, not one branch of a sum. When several alternatives can consume the same symbol, they stay as separate elements, and $smartC$ is applied to each independently. One strand may finish the left operand while another still carries a non-empty residual in front of s . The constructor is therefore applied independently to each residual produced by $pd(r, x)$, preserving the distinct continuations of the concatenation without introducing redundant $\varepsilon \cdot s$ expressions.

In Rust. Figure 3.7 is *smartC*. *smart_seq* takes *r* by value and *s* by reference, matching (3.18). The residual *r'* comes from the recursive derivative and is consumed, while *s* must survive into the result. The $r' = \varepsilon$ branch clones only *s* and drops ε , so $\varepsilon \cdot s$ is never built.

```
pub fn smart_seq(r: Regex, s: &Regex) -> Regex {
    match r {
        Regex::Eps => s.clone(),
        r => Regex::seq(r, s.clone()),
    }
}
```

Figure 3.7: *smart_seq*, the smart constructor *smartC*

Example: No growth under *pd*. Consider $(x + y)^*$ against *xx* under *pd*.

Step 1. P_* applies with star body $r = x + y$:

$$\begin{aligned} pd((x + y)^*, \mathbf{x}) &= \{ \text{smartC}(r', (x + y)^*) \mid r' \in pd(x + y, \mathbf{x}) \} & P_* \\ &= \{ \text{smartC}(r', (x + y)^*) \mid r' \in \{\varepsilon\} \} & P_+, P_{\text{lit}} \text{ twice} \\ &= \{(x + y)^*\} & \text{SC}, r' = \varepsilon. \end{aligned} \quad (3.19)$$

$pd(x + y, \mathbf{x}) = \{\varepsilon\} \cup \{\} = \{\varepsilon\}$ by P_+ , since *x* matches the left literal and misses the right, and *smartC* absorbs the ε straight back into $(x + y)^*$.

Step 2. The set is $\{(x + y)^*\}$, so deriving by the second *x* repeats the computation:

$$pd((x + y)^*, \mathbf{x}, \mathbf{x}) = \{(x + y)^*\}.$$

Since $(x + y)^*$ is nullable by N_* , the final partial derivative accepts ε , so $\mathbf{xx} \in L((x + y)^*)$.

For comparison, applying the Brzozowski derivative to the same expression gives the intermediate alternation in (3.8), $(\emptyset + \emptyset) \cdot (x + y)^* + (\varepsilon + \emptyset) \cdot (x + y)^*$. Only after simplification in (3.12) does this reduce to $(x + y)^*$. Under *pd*, the residual set is $\{(x + y)^*\}$ after both steps, since $P_\emptyset$ and P_ε produce no residual at all. So redundant terms are avoided at construction time rather than built and simplified afterwards.

The next example shows where more than one residual survives at once.

Running example: $r_{01} = (0 + 1)^* \cdot 0 \cdot 1$. Derive r_{01} against $w = 101$, tracking the residual set. The concatenations bind as in Section 3.2, with $(r, s) = ((0 + 1)^* \cdot 0, 1)$ at the outer node and $(r, s) = ((0 + 1)^*, 0)$ at the inner node; the star body is $0 + 1$. The derivation proceeds from the inside outward.

1. **Derive by 1.** At the star body, P_+ distributes the derivative over two alternatives:

$$\begin{aligned} pd(0+1, 1) &= pd(0, 1) \cup pd(1, 1) & P_+, r = 0, s = 1 \\ &= \{\} \cup \{\varepsilon\} & P_{\text{lit}}, \text{ both branches} \\ &= \{\varepsilon\}. \end{aligned} \quad (3.20)$$

The star rule then applies *smartC* to the resulting residual:

$$\begin{aligned} pd((0 + 1)^*, 1) &= \{ \text{smartC}(r', (0 + 1)^*) \mid r' \in \{\varepsilon\} \} & P_*, r = 0 + 1 \\ &= \{(0 + 1)^*\} & \text{SC}, r' = \varepsilon. \end{aligned} \quad (3.21)$$

At the middle concatenation, the left operand is nullable by $\mathbf{N}_*$, so the nullable case contributes both the derivatives through the left and right operand:

$$\begin{aligned}
pd((0+1)^* \cdot 0, 1) &= \{ smartC(r', 0) \mid r' \in pd((0+1)^*, 1) \} \cup pd(0, 1) && \mathbf{P}_{seq}^n \\
&= \{ smartC((0+1)^*, 0) \} \cup \{ \} && \text{above; } \mathbf{P}_{lit} \\
&= \{ (0+1)^* \cdot 0 \} && \mathbf{SC}, r' \neq \varepsilon.
\end{aligned} \tag{3.22}$$

At the outer node, the left operand is not nullable, so only one branch contributes:

$$\begin{aligned}
pd(r_{01}, 1) &= \{ smartC(r', 1) \mid r' \in \{ (0+1)^* \cdot 0 \} \} && \mathbf{P}_{seq}, r = (0+1)^* \cdot 0, s = 1 \\
&= \{ (0+1)^* \cdot 0 \cdot 1 \} = \{ r_{01} \} && \mathbf{SC}.
\end{aligned} \tag{3.23}$$

The first symbol leaves $\{r_{01}\}$, returning to the starting expression since the consumed 1 belongs to the leading star.

2. **Derive by 0.** The current set is still $\{r_{01}\}$. As before, $pd((0+1)^*, 0) = \{(0+1)^*\}$. At the middle concatenation, however, the right operand is now the matching literal 0, so $\mathbf{P}_{lit}$ contributes $\{\varepsilon\}$:

$$\begin{aligned}
pd((0+1)^* \cdot 0, 0) &= \{ smartC((0+1)^*, 0) \} \cup pd(0, 0) && \mathbf{P}_{seq}^n, r = (0+1)^*, s = 0 \\
&= \{ (0+1)^* \cdot 0 \} \cup \{ \varepsilon \} && \mathbf{P}_{lit}, \text{ since } 0 = 0 \\
&= \{ (0+1)^* \cdot 0, \varepsilon \}.
\end{aligned} \tag{3.24}$$

The outer concatenation applies $smartC$ separately to both residuals:

$$\begin{aligned}
pd(r_{01}, 1, 0) &= \{ smartC(r', 1) \mid r' \in \{ (0+1)^* \cdot 0, \varepsilon \} \} && \mathbf{P}_{seq}, s = 1 \\
&= \{ (0+1)^* \cdot 0 \cdot 1, 1 \} && \mathbf{SC} \\
&= \{ r_{01}, 1 \}.
\end{aligned} \tag{3.25}$$

The two residuals are the possible strands after 10: r_{01} keeps both symbols in the leading star, while 1 means the 0 began the final 01. Since $smartC(\varepsilon, 1) = 1$, the latter is represented directly as 1.

3. **Derive by 1.** The two residuals are derived independently and their results are unioned:

$$\begin{aligned}
pd(r_{01}, 1, 0, 1) &= pd(r_{01}, 1) \cup pd(1, 1) && \text{union over the set} \\
&= \{ r_{01} \} \cup \{ \varepsilon \} && \text{step 1; } \mathbf{P}_{lit} \\
&= \{ r_{01}, \varepsilon \}.
\end{aligned} \tag{3.26}$$

The final set contains ε , so one surviving residual is nullable and $101 \in L(r_{01})$. This agrees with the conclusion reached by the Brzozowski derivative in (3.15).

For $w = 10$, the first two steps leave $\{r_{01}, 1\}$, which represents the same alternatives as $((0+1)^* \cdot 0 + \varepsilon) \cdot 1$ in (3.14). Either the trailing 01 remains to be matched, or the consumed 0 was its first symbol. Neither residual is nullable, so $10 \notin L(r_{01})$.

Finiteness. For a fixed regular expression r , the set of reachable partial derivatives is finite and bounded by $\|r\| + 1$, where $\|r\|$ denotes the number of literal occurrences in r [Antimirov, 1996, Theorem 3.4, p. 309]. For r_{01} , $\|r_{01}\| = 4$ (the four literals: 0, 1, 0, 1), so the bound is $4 + 1 = 5$ distinct residuals across all possible inputs. The example with $w = 101$ reaches only three of them: $\{r_{01}\}$, $\{r_{01}, 1\}$, and $\{r_{01}, \varepsilon\}$. Other residuals in the theoretical set are reachable only by different input sequences.

For ordinary derivatives, finiteness requires identifying equivalent expressions [Brzozowski, 1964, Theorem 4.3], yielding an exponential bound $d_R \leq 2^{d_P} - 1$. Simplification rules eliminate only units and redundant $\emptyset$ expressions, not all equivalent forms. The linear bound for partial derivatives applies directly without simplification.

The matcher. A *frontier* of residuals is updated one symbol at a time by taking pd of every element and unioning the results. The frontier F_i represents all residuals reachable after consuming the first i input symbols—every possible state the computation could be in. For frontier F_i after i symbols,

$$F_0 = \{r\}, \quad F_i = \bigcup_{r' \in F_{i-1}} pd(r', x_i) \quad (1 \leq i \leq n). \quad (3.27)$$

The matcher accepts when some residual in the final frontier is nullable. Where Brzozowski's $r_0 \rightarrow r_1 \rightarrow \dots \rightarrow r_n$ traces one path through a lazily constructed DFA, $F_0 \rightarrow F_1 \rightarrow \dots \rightarrow F_n$ traces all states an NFA explores simultaneously, holding each state at once rather than committing to a single residual expression.

In Rust. Figure 3.8 implements (3.27) using a `HashSet<Regex>` frontier, where `Regex`'s `Hash` and `Eq` implementations allow residuals from different paths to collapse. Where the Brzozowski matcher holds one `Regex`, this holds a set. The early return when the frontier empties follows from construction: no later derivative produces residuals from an empty set. The final `any(nullable)` checks whether some residual is nullable. The matcher trades a single growing expression for a set of small ones that never need simplification.

```
pub fn match_pderiv(input: &str, r: &Regex) -> bool {
    let mut states: HashSet<Regex> = HashSet::new();
    states.insert(r.clone());
    for c in input.chars() {
        let mut next: HashSet<Regex> = HashSet::new();
        for state in &states {
            next.extend(pderiv(state, c));
        }
        if next.is_empty() { return false; }
        states = next;
    }
    states.iter().any(nullable)
}
```

Figure 3.8: The partial-derivative matcher.

4 Matching versus Parsing

Chapter 3 established matching with a Boolean answer: whether $w \in L(r)$. A matcher therefore answers only whether the input is accepted by the regular expression. Parsing goes beyond this membership question by identifying the derivation for the match.

This distinction is important because a regular expression can admit more than one way of matching the same word. A parser must therefore represent the choices made during a match. These choices can be represented by *parse trees*, which provide structured witnesses for membership in the language.

4.1 Matching Ambiguity

Consider the regular expression

$$r = (a + ab)(b + \varepsilon)$$

and the input word

$$w = \mathbf{ab}.$$

The matcher can establish that $w \in L(r)$ using the matching procedures of Chapter 3. There are, however, two ways in which the match can be obtained. In the first, the left subexpression matches **a** and the right subexpression matches **b**:

$$\underbrace{a}_{a+ab} \quad \underbrace{b}_{b+\varepsilon}.$$

In the second, the left subexpression matches the entire input and the right subexpression matches the empty word:

$$\underbrace{\mathbf{ab}}_{a+ab} \quad \underbrace{\varepsilon}_{b+\varepsilon},$$

Both choices therefore establish the same membership result,

$$\mathbf{ab} \in L((a + ab)(b + \varepsilon)),$$

but assign the input differently to the subexpressions of r . The Boolean result of matching does not distinguish these cases.

Parsing makes this additional structure explicit. A parser must represent which alternatives were chosen and which parts of the input were consumed by the corresponding subexpressions. A *parse tree* provides a structured witness for such a derivation, recording the choices made by the expression and the corresponding consumption of the input.

4.2 Parse Trees as Matching Evidence

Sulzmann and Lu view regular expressions as types and parse trees as their values [Sulzmann and Lu, 2014]. The judgment $\vdash v : r$ states that v is a valid parse tree for r and records the structural choices made when matching r :

$$\begin{array}{c}
 \vdash () : \varepsilon \quad (\mathsf{T}_\varepsilon) \\
 \\
 \frac{l \in \Sigma}{\vdash l : l} \quad (\mathsf{T}_{\text{lit}}) \\
 \\
 \frac{\vdash v_1 : r_1 \quad \vdash v_2 : r_2}{\vdash (v_1, v_2) : r_1 r_2} \quad (\mathsf{T}_{\text{seq}}) \\
 \\
 \frac{\vdash v_1 : r_1}{\vdash \text{Left } v_1 : r_1 + r_2} \quad (\mathsf{T}_+^L) \\
 \\
 \frac{\vdash v_2 : r_2}{\vdash \text{Right } v_2 : r_1 + r_2} \quad (\mathsf{T}_+^R) \\
 \\
 \frac{}{\vdash [] : r^*} \quad (\mathsf{T}_*^0) \\
 \\
 \frac{\vdash v : r \quad \vdash vs : r^*}{\vdash (v : vs) : r^*} \quad (\mathsf{T}_*)
 \end{array} \tag{4.1}$$

The rules correspond to the constructors of regular expressions. A concatenation uses a pair of parse trees, an alternation records the chosen branch with a tagged injection, and a Kleene star records its iterations as a list.

In Rust. The definitions above translate directly to Rust. Figure 4.1 shows the parse-tree type, with one variant for each constructor in (4.1). The two star rules share a single **Star** variant, represented by a **Vec**.

```

#[derive(Debug, Clone, PartialEq, Eq)]
pub enum ParseTree {
    Empty,                                // ()
    Char(char),                           // l
    Pair(Box<ParseTree>, Box<ParseTree>), // (v1, v2)
    Left(Box<ParseTree>),                  // Left v
    Right(Box<ParseTree>),                  // Right v
    Star(Vec<ParseTree>),                   // [v1, ..., vn]
}

```

Figure 4.1: The parse-tree type: The variants correspond to the constructors. The two star rules share the **Star** variant.

The correspondence is one to one except for the star case: T_ε , T_{lit} , T_{seq} , T_+^L , and T_+^R correspond to **Empty**, **Char**, **Pair**, **Left**, and **Right**, respectively. The empty and non-empty star rules are both represented by **Star**, with $[]$ represented by **Star**(**vec!** $[]$).

The recursive variants use **Box<ParseTree>** because Rust requires recursive types to have a statically known size. **Star** instead uses **Vec<ParseTree>**, which already provides indication through a fixed-size vector handle.

Derived traits. `PartialEq` and `Eq` allow direct comparison of parse trees, while `Hash` is omitted because parse trees are not used as keys for deduplication.

The typing judgment describes the structure of a match, but a parse tree must also be related to the word that it matches. For this purpose, they define a *flattening* function $|v|$ that maps a parse tree to the word represented by its structure:

$$\begin{aligned} |()| &= \varepsilon, & |(v_1, v_2)| &= |v_1||v_2|, \\ |l| &= l, & |Left\ v| &= |v|, \\ |[]| &= \varepsilon, & |Right\ v| &= |v|, \\ & & |v : vs| &= |v||vs|. \end{aligned} \tag{4.2}$$

The flattening function connects parse trees to the language of a regular expression:

$$L(r) = \{ |v| \mid \vdash v : r \}. \tag{4.3}$$

Thus, a parse tree v provides a structured *witness* for membership in $L(r)$, recording both the match and the structural choices by which r produces the word.

In Rust. The function takes `&ParseTree` rather than consuming the tree, so it can be inspected, printed, and compared after flattening. The star case similarly borrows its elements through `vs.iter()`. Keeping flattening separate from tree construction also provides an independent parse validation. A tree for w must satisfy $|v| = w$, as required by (4.5). Flattening is implemented directly from (4.2):

```
pub fn flatten(v: &ParseTree) -> String {
    match v {
        ParseTree::Empty      => String::new(),
        ParseTree::Char(c)    => c.to_string(),
        ParseTree::Pair(v1, v2) =>
            format!("{}", flatten(v1), flatten(v2)),
        ParseTree::Left(v)    => flatten(v),
        ParseTree::Right(v)   => flatten(v),
        ParseTree::Star(vs)   => vs.iter().map(flatten).collect(),
    }
}
```

Figure 4.2: The `flatten` function: extract the matched word from a parse tree.

Unambiguous example. Consider $r = ab$ and $w = \mathbf{ab}$. The sequence rule requires a parse tree for a followed by a parse tree for b . Since each subexpression is a literal, the literal rule gives the only possible derivation:

$$\frac{\frac{a \in \Sigma}{\vdash a : a} (\mathsf{T}_{\text{lit}}) \quad \frac{b \in \Sigma}{\vdash b : b} (\mathsf{T}_{\text{lit}})}{\vdash (a, b) : ab} (\mathsf{T}_{\text{seq}}) \tag{4.4}$$

The parse tree is $v = (a, b)$ and flattening recovers the input word: $|(a, b)| = |a| |b| = \mathbf{ab}$.

4.3 Disambiguation

The typing judgment identifies all valid parse trees for a given regular expression, while the flattening function determines which word each tree represents. So for a regular expression r and word w , a parse result must satisfy:

$$\vdash v : r \quad \text{and} \quad |v| = w. \quad (4.5)$$

When several parse trees satisfy these conditions, the typing judgment does not determine which one should be returned. A *disambiguation policy* specifies how this choice is made. Two policies commonly considered in the literature are *Greedy* and *POSIX*:

- *Greedy* resolves choices by the order in which the expression offers them, preferring the earlier alternative and preferring a further iteration to stopping, whatever each consumes [Frisch and Cardelli, 2004].
- *POSIX* resolves choices by subpattern match length from left to right, with ties resolved by the expression order [IEEE and The Open Group, 2017].

The precise definitions of these policies are given in the corresponding literature; see Frisch and Cardelli [2004] and Sulzmann and Lu [2014] for formal treatments. Instead, the examples below show how the policies distinguish between valid parse trees.

Ambiguity in concatenation. For $r_1 = (a + ab)(b + \varepsilon)$ and $w = \mathbf{ab}$, there are two possible matches, corresponding to two distinct parse trees.

The first consumes $\mathbf{ab}$ with the second alternative of $a + ab$ and then chooses ε :

$$\frac{\frac{\vdash a : a \quad \vdash b : b}{\vdash (a, b) : ab}}{\vdash \text{Right}(a, b) : a + ab} \quad \frac{\vdash () : \varepsilon}{\vdash \text{Right}() : b + \varepsilon} \Rightarrow \vdash (\text{Right}(a, b), \text{Right}()) : (a + ab)(b + \varepsilon). \quad (4.6)$$

Thus,

$$v_1 = (\text{Right}(a, b), \text{Right}()), \quad |v_1| = \mathbf{ab}. \quad (4.7)$$

The second matches $\mathbf{a}$ and $\mathbf{b}$ using the first alternatives of $a + ab$ and $b + \varepsilon$:

$$\frac{\vdash a : a}{\vdash \text{Left } a : a + ab} \quad \frac{\vdash b : b}{\vdash \text{Left } b : b + \varepsilon} \Rightarrow \vdash (\text{Left } a, \text{Left } b) : (a + ab)(b + \varepsilon). \quad (4.8)$$

Thus,

$$v'_1 = (\text{Left } a, \text{Left } b), \quad |v'_1| = \mathbf{ab}. \quad (4.9)$$

Hence $v_1 \neq v'_1$ are valid parse trees for the same expression and word. POSIX selects v_1 because the leftmost subpattern matches the longer string $\mathbf{ab}$, while Greedy selects v'_1 because its left alternative comes first.

Ambiguity in repetition. A similar ambiguity arises from repetition. Consider $r_2 = (a + b + ab)^*$ against $w = \mathbf{ab}$, with the alternation nested as $a + (b + ab)$. The word can either be consumed by one iteration using the ab alternative or by two iterations consuming $\mathbf{a}$ and $\mathbf{b}$ separately.

The first match is

$$\frac{\frac{\frac{\vdash a : a \quad \vdash b : b}{\vdash (a, b) : ab}}{\vdash \text{Right}(a, b) : b + ab}}{\vdash \text{Right Right}(a, b) : a + (b + ab)} \Rightarrow \vdash [\text{Right Right}(a, b)] : (a + (b + ab))^*. \quad (4.10)$$

Thus,

$$v_2 = [\text{Right Right}(a, b)], \quad |v_2| = \mathbf{ab}. \quad (4.11)$$

The second match uses two iterations:

$$\frac{\vdash a : a}{\vdash \text{Left } a : a + (b + ab)} \quad \frac{\frac{\vdash b : b}{\vdash \text{Left } b : b + ab}}{\vdash \text{Right Left } b : a + (b + ab)} \quad (4.12)$$

$$\Rightarrow \vdash [\text{Left } a, \text{Right Left } b] : (a + (b + ab))^*.$$

Thus,

$$v'_2 = [\text{Left } a, \text{Right Left } b], \quad |v'_2| = \mathbf{ab}. \quad (4.13)$$

Here POSIX selects the single-iteration tree v_2 , since the star's first iteration can match both characters through the ab alternative. Greedy selects the two-iteration tree v'_2 , since the alternative a comes first and consuming only $\mathbf{a}$ leaves $\mathbf{b}$ for a second iteration.

The examples show that the same regular expression and input can admit multiple valid parse trees, with the selected tree depending on the disambiguation policy. The next two chapters apply these ideas to derivative-based parsing and compare the resulting parser implementations.

5 Derivative-Based Parser

Chapter 3 showed how a Brzozowski-derivative matcher determines whether $w \in L(r)$ and Chapter 4 showed that a matching word may admit multiple valid parse trees. A parser must therefore select a single parse tree according to a disambiguation policy.

Following Sulzmann and Lu [Sulzmann and Lu, 2014], this chapter shows two methods for constructing POSIX parse trees using derivatives. The direct construction performs a forward derivative pass followed by backward tree reconstruction. Meanwhile, the bit-coded construction records parse-tree information during the forward pass and can reconstruct the tree without a separate backward pass. Both methods produce identical POSIX trees, and each algorithmic step is accompanied by its Rust implementation.

5.1 The Two-Pass Approach

Consider $r = a \cdot b$ with input $w = \mathbf{ab}$. This expression is unambiguous and has exactly one parse tree, so the question is whether it can be recovered from the derivation sequence:

$$\begin{aligned} a \cdot b &\xrightarrow{a} \varepsilon \cdot b \\ \varepsilon \cdot b &\xrightarrow{b} \phi \cdot b + \varepsilon. \end{aligned} \tag{5.1}$$

The final expression is nullable, confirming $w \in L(r)$, as established by the derivative construction in Chapter 3. However, no consumed character survives in the final expression, so the parse tree (a, b) cannot be recovered directly from $\phi \cdot b + \varepsilon$.

The derivation sequence itself already contains the information needed to reconstruct the tree. Each step transforms the expression by consuming one character. Reversing these steps reinserts the characters into a parse tree, so it can be recovered by walking the same sequence backward. Figure 5.1 shows this process, with the forward pass showing the derivation sequence and the backward pass showing the corresponding tree reconstruction.

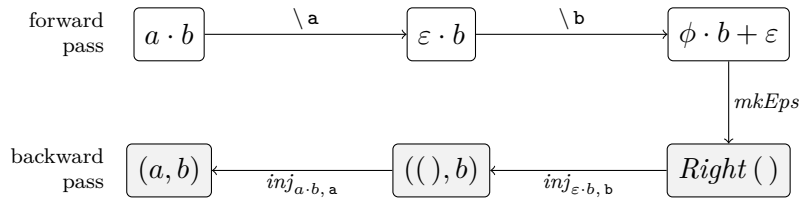


Figure 5.1: Direct construction with forward and backward pass: Forward derivative sequence (top row), Backward parse-tree reconstruction via *mkEps* and *inj* (bottom row)

The two-pass pipeline. The backward pass reconstructs the parse tree by walking the derivation sequence in reverse. *mkEps* builds a parse tree for the empty word from the final nullable expression. Each injection step *inj* then reinserts one character, moving right to left. After the first injection, the tree is $((), b)$, where $()$ holds the position for a . After the second injection, that position receives a and the tree (a, b) is complete.

The example above is unambiguous and therefore does not address the disambiguation problem. The following section presents the standard derivative-based parser with its reconstruction operations, including *mkEps* and *inj*, while showing how ambiguity is resolved using the POSIX disambiguation policy.

5.2 The Standard Derivative-Based Parser

The direct construction uses two passes. The forward pass applies *deriv* 3.3 and *nullable* 3.2 from Chapter 3 to process the input. The backward pass applies two new functions, *mkEps* and *inj*, to reconstruct a parse tree 4.1 from the derivation sequence.

Unlike the matcher from Section 3.2, the direct construction does not simplify expressions during the forward pass. Simplification is deliberately omitted because the backward reconstruction depends on the explicit structure of the expressions. Removing nodes during simplification would discard information required for tree reconstruction.

5.2.1 Building the Empty-Word Tree

The function *mkEps* takes a nullable expression and constructs a parse tree for the empty word by recursively traversing the expression's structure. This function serves as the base case for the backward pass. Once all input has been consumed and the final expression is known to be nullable, *mkEps* produces the initial parse tree that subsequent injection steps will extend and refine.

$$\begin{aligned}
mkEps_{\varepsilon} &= () & (E_{\varepsilon}) \\
mkEps_{r^*} &= [] & (E_*) \\
mkEps_{r_1 r_2} &= (mkEps_{r_1}, mkEps_{r_2}) & (E_{\text{seq}}) \\
mkEps_{r_1 + r_2} &= \begin{cases} \text{Left } mkEps_{r_1} & \varepsilon \in L(r_1) \\ \text{Right } mkEps_{r_2} & \text{otherwise} \end{cases} & \begin{matrix} (E_+^L) \\ (E_+^R) \end{matrix}
\end{aligned} \tag{5.2}$$

The first three rules follow directly from the structure of the expression. The expression ε has a single parse tree, a star matching the empty word represents zero iterations, and a concatenation matches ε only when both factors do. No rules are required for $\emptyset$ or literals, since neither can match the empty word.

The alternation rule incorporates the POSIX disambiguation policy. If both r_1 and r_2 are nullable, both alternatives match the empty word, so the longest-match criterion does not distinguish them. The left alternative is therefore selected by the guard $\varepsilon \in L(r_1)$. This corresponds to the left-to-right priority required by the POSIX policy when the alternatives have equal match length. Sulzmann and Lu show that this local choice leads to the POSIX parse tree globally [Sulzmann and Lu, 2014].

In Rust. The recursive definition of *mkEps* (5.2) is implemented directly in Rust, as shown in Figure 5.2 and follows the same cases for each expression form.

```
pub fn mk_eps(r: &Regex) -> ParseTree {
  match r {
    Regex::Eps => ParseTree::Empty,
    Regex::Star(_) => ParseTree::Star(Vec::new()),
    Regex::Seq(r1, r2) => ParseTree::Pair(
      Box::new(mk_eps(r1)), Box::new(mk_eps(r2))),
    Regex::Alt(r1, r2) => if nullable(r1) {
      ParseTree::Left(Box::new(mk_eps(r1)))
    } else {
      ParseTree::Right(Box::new(mk_eps(r2)))
    },
    Regex::Lit(c) => panic!("mk_eps called on Lit('{ }')", c),
    Regex::Phi => panic!("mk_eps called on Phi"),
  }
}
```

Figure 5.2: *mk_eps*: construct a parse tree for the empty word.

The expression r is borrowed rather than consumed because the backward pass reads it again during injection. The alternation arm encodes the POSIX decision as a conditional. r_2 is examined only when `nullable(r1)` is false. Since the input is the empty word, there are no lengths to compare, so the longest-match criterion does not apply. The final two arms panic rather than return arbitrary trees, encoding the precondition that neither ϕ nor a literal is nullable.

5.2.2 Reinserting Characters via Injection

The function *inj* reverses a single derivation step. Given a parse tree for $r \backslash x$, it reconstructs a parse tree for r by reinserting the consumed character x .

$$\begin{aligned}
inj_{x \backslash x}() &= x & (I_{\text{lit}}) \\
inj_{(r_1+r_2) \backslash x}(Left \ v_1) &= Left \ (inj_{r_1 \backslash x} \ v_1) & (I_{+}^L) \\
inj_{(r_1+r_2) \backslash x}(Right \ v_2) &= Right \ (inj_{r_2 \backslash x} \ v_2) & (I_{+}^R) \\
inj_{r^* \backslash x}(v, vs) &= (inj_{r \backslash x} \ v) : vs & (I_{*}) \\
inj_{(r_1 r_2) \backslash x}(v_1, v_2) &= (inj_{r_1 \backslash x} \ v_1, \ v_2) & (I_{\text{seq}}), \ n(r_1) = \text{false} \\
inj_{(r_1 r_2) \backslash x}(Left(v_1, v_2)) &= (inj_{r_1 \backslash x} \ v_1, \ v_2) & (I_{\text{seq}}^L), \ n(r_1) = \text{true} \\
inj_{(r_1 r_2) \backslash x}(Right \ v_2) &= (mkEps_{r_1}, \ inj_{r_2 \backslash x} \ v_2) & (I_{\text{seq}}^R), \ n(r_1) = \text{true}
\end{aligned} \tag{5.3}$$

The first four rules follow directly from the corresponding expression constructors. A literal derived to ε is reconstructed as its consumed character. For an alternation, the incoming tag identifies the branch containing the derived expression. For a star, the first component of the incoming tree represents the current iteration, which is reconstructed and prepended to the remaining iterations.

The concatenation rules cover the two cases of D_{seq} . When r_1 is not nullable, no alternation is introduced, and the character is reconstructed within the left component. When r_1 is nullable, the derivative introduces an alternation whose tag records whether the character was consumed by r_1 or passed to r_2 . In the latter case, r_1 is absent from the derivation sequence, so its parse tree is reconstructed using $mkEps_{r_1}$.

```

pub fn inject(r: &Regex, l: char, v: ParseTree) -> ParseTree {
  match r {
    Regex::Lit(c) => { assert!(*c == l);
      ParseTree::Char(l) }
    Regex::Alt(r1, r2) => match v {
      ParseTree::Left(v1) =>
        ParseTree::Left(Box::new(inject(r1, l, *v1))),
      ParseTree::Right(v2) =>
        ParseTree::Right(Box::new(inject(r2, l, *v2))),
      _ => panic!("inject/Alt: want Left|Right"),
    },
    Regex::Star(r1) => match v {
      ParseTree::Pair(v1, vs) => {
        let mut it = vec![inject(r1, l, *v1)];
        if let ParseTree::Star(rest) = *vs { it.extend(rest); }
        ParseTree::Star(it)
      }
      _ => panic!("inject/Star: want Pair"),
    },
    Regex::Seq(r1, r2) => if !nullable(r1) {
      match v {
        ParseTree::Pair(v1, v2) => ParseTree::Pair(
          Box::new(inject(r1, l, *v1)), v2),
        _ => panic!("inject/Seq: want Pair"),
      }
    } else {
      match v {
        ParseTree::Left(vp) => match *vp {
          ParseTree::Pair(v1, v2) => ParseTree::Pair(
            Box::new(inject(r1, l, *v1)), v2),
          o => panic!("inject/SeqL: want Pair, got {:?}", o),
        },
        ParseTree::Right(v2) => ParseTree::Pair(
          Box::new(mk_eps(r1)),
          Box::new(inject(r2, l, *v2))),
        _ => panic!("inject/Seq: want Left|Right"),
      }
    },
    Regex::Eps | Regex::Phi => panic!("inject called on Eps/Phi"),
  }
}

```

Figure 5.3: `inject`: reinsert a character into a parse tree.

In Rust. `inject(r, l, v)` takes three arguments because *inj* is indexed by both an expression and a character. Figure 5.3 shows the complete implementation.

The function matches on `r` first, then `v`, because (5.3) indexes every rule by expression first. A tag like `Left` means different things depending on context. Under an alternation, it selects the left disjunct. Under a nullable concatenation, it indicates that the derivative continued inside the left operand rather than skipping to the right. Matching `v` before `r` would lose this context.

The expression `r` is borrowed and read only, as the tree `v` is consumed and decomposed. In the concatenation arms where the right operand was not derived, `v2` moves directly into the new pair with no copy, since (5.3) leaves v_2 unchanged. Only the `Right` arm of concatenation builds new content: the derivative skipped r_1 entirely, so nothing from the forward pass exists for its half of the pair. Therefore, `mk_eps(r1)` synthesizes it.

Calling `nullable(r1)` here recomputes what the forward pass already decided when choosing between D_{seq} and D_{seq}^n . Caching this information would require storing a separate data structure alongside the expression sequence, adding overhead.

5.2.3 Parser

The parser composes `mkEps` and `inject` by storing all expressions from the forward pass, then reconstructing the tree by walking backward. Figure 5.4 implements this in a loop. A recursive implementation would use the call stack to hold the expression sequence, but stacks have fixed size while a `Vec` grows dynamically on the heap, allowing the loop form to handle longer inputs without overflow.

```
pub fn parse_deriv_std_loop(input: &str, r: &Regex)
    -> Option<ParseTree>
{
    let chars: Vec<char> = input.chars().collect();
    let mut expressions = Vec::with_capacity(chars.len() + 1);
    expressions.push(r.clone());
    for &c in &chars {
        expressions.push(deriv(expressions.last().unwrap(), c));
    }
    if !nullable(expressions.last().unwrap()) { return None; }
    let mut tree = mk_eps(expressions.last().unwrap());
    for i in (0..chars.len()).rev() {
        tree = inject(&expressions[i], chars[i], tree);
    }
    Some(tree)
}
```

Figure 5.4: Standard derivative-based parser. Forward pass builds the expression sequence; Backward pass reconstructs the parse tree via `mkEps` and `inject`.

Every parser presented returns `Option<ParseTree>`, which allows direct comparison of outputs. The `flatten` function 4.2 provides an independent correctness check. As shown in Section 4.2, a parse tree is a valid witness for membership only if $flatten(v) = w$.

5.2.4 Worked Example

Consider input `ab` and the regular expression $r_0 = (a \mid ab)(b \mid \varepsilon)$.

Forward pass. The two input symbols are consumed by successive derivatives:

$$\begin{aligned} r_0 = (a + ab)(b + \varepsilon) &\xrightarrow[\text{D}_{\text{seq}}]{a} r_1 = (\varepsilon + \varepsilon b)(b + \varepsilon) \\ &\xrightarrow[\text{D}_{\text{seq}}^n]{b} r_2 = \underbrace{(\phi + (\phi b + \varepsilon))(b + \varepsilon)}_X + \underbrace{(\varepsilon + \phi)}_Y. \end{aligned} \quad (5.4)$$

Both X and Y are nullable. So $mkEps$ chooses the left disjunct by E_+^L , which causes the first factor to match `ab` rather than `a`, implementing POSIX's leftmost-longest rule:

$$mkEps(r_2) = mkEps(X + Y) \xrightarrow{E_+^L} \text{Left } mkEps(X). \quad (5.5)$$

Backward pass. Injection reverses both derivatives. Starting from $mkEps(r_2)$, undoing `b` applies l_{seq}^L , l_+^R , and l_{seq}^R ; undoing `a` then applies l_{seq} twice:

$$\begin{aligned} v_2 &= mkEps(r_2) = \text{Left}(\text{Right } \text{Right } (), \text{Right } ()) \\ v_1 &= \text{inj}_{r_1 \setminus b}(v_2) = (\text{Right } (), b, \text{Right } ()) \\ v_0 &= \text{inj}_{r_0 \setminus a}(v_1) = (\text{Right } (a, b), \text{Right } ()). \end{aligned} \quad (5.6)$$

The diagnostic trace 5.5 corresponds to the derivation above and is generated by traced variants of `deriv` 3.3, `nullable` 3.2, `mkEps` 5.2, and `inj` 5.3.

FORWARD PASS (Derivatives)	NULLABILITY CHECK
-----	-----
<code>r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))</code>	<code>nullable(r2) = true -> proceed to backward pass</code>
 Step 1: <code>d(r0, 'a')</code> rule: <code>d(r1*r2, 'a')</code> [<code>nullable(r1)=false</code>] = <code>d(r1, 'a')*r2</code>	 BACKWARD PASS (<code>mkEps + inject</code>) -----
<code>r1 = Seq(Alt(Eps, Seq(Eps, Lit('b'))), Alt(Lit('b'), Eps))</code> <code>nullable(r1) = true [OK]</code>	<code>mkEps(r2): mkEps(r1 + r2): nullable(r1)=true -> Left(mkEps(r1))</code>
 Step 2: <code>d(r1, 'b')</code> rule: <code>d(r1*r2, 'b')</code> [<code>nullable(r1)=true</code>] = <code>d(r1, 'b')*r2</code> + <code>d(r2, 'b')</code>	 <code>v2 = Left (Right Right (), Right ())</code>
<code>r2 = Alt(Seq(Alt(Phi, Alt(Seq(Phi, Lit('b'))), Eps)), Alt(Lit('b'), Eps)), Alt(Eps, Phi))</code> <code>nullable(r2) = true [OK]</code>	 <code>inject(r1, 'b', v2): -> (Right (), b, Right ()) v1 = (Right (), b, Right ())</code>
	 <code>inject(r0, 'a', v1): -> (Right (a, b), Right ()) v0 = (Right (a, b), Right ())</code>

Figure 5.5: Diagnostic trace of the two-pass construction.

The final tree represents the ab alternative and ε . The function `flatten` 4.2 validates it:

$$\text{flatten}(v_0) = a \cdot b \cdot \varepsilon = \text{ab}. \quad (5.7)$$

RESULT

```
-----
Parse tree:      (Right (a, b), Right ())
Flattened:       "ab"
Expressions stored: 3 (r0 -> r1 -> r2)
```

Figure 5.6: Final parse tree and validation via `flatten`.

This construction stores three intermediate expressions. The next section presents an alternative implementation that avoids storing these expressions.

5.3 The Bit-Coded Approach

The direct construction cannot reconstruct the parse tree until the input has been exhausted, because *inj* requires each preceding expression r_{i-1} . Sulzmann and Lu’s bit-coded construction [Sulzmann and Lu, 2014] removes this dependency by recording the information needed for reconstruction during the forward pass. Figure 5.7 shows this on pattern a^* and input "aaa". Unlike Figure 5.1, the derivation remains a single forward sequence, while a growing bit-prefix records the choices needed to reconstruct the parse tree.

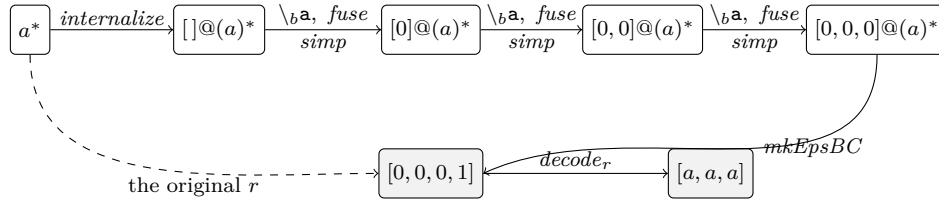


Figure 5.7: The bit-coded construction on a^* and input "aaa". The bit-prefix records reconstruction choices, while *decode_r* uses the original r to recover the parse tree.

The bit-coded pipeline. Figure 5.7 shows the four main operations of the bit-coded method. *internalize* converts the input pattern r into an annotated expression, tagging each alternation branch with an initial bit. *fuse* prepends the bit-lists to the annotations of existing nodes, to allow this information to be carried between subexpressions. At each character, the annotated expression is derived using a bit-threaded derivative operation, then simplified by *simp*.

Unlike the direct construction, which forbids simplification to preserve nodes that backward reconstruction requires, the bit-coded method allows it because the bits carry the information a backward pass would need. It is applied after each derivative step to control expression growth.

Once the input is consumed, *mkEpsBC* extracts the accumulated bits from the final expression, using the same left-preference rule at alternations as *mkEps* does in the direct construction. The function *decode_r* then interprets these bits using the original pattern r to reconstruct the parse tree, reading off which branch was taken at each choice point.

5.4 The Bit-Coded Derivative-Based Parser

Building the parse tree information during the forward pass extends the derivative computation with annotations that carry bit-lists through each derivation step. The bit-threaded derivative $ri \setminus_{bl}$ propagates these annotations, while $mkEpsBC$ and $decode_r$ use the final bit-list to reconstruct the `ParseTree`.

5.4.1 Annotated Expressions

An annotated expression ri extends r by associating a bit-list bs with every node except $\emptyset$. The bit-list records the reconstruction choices made during the forward pass, such as which branch of an alternation or which iteration of a star is selected.

$$ri ::= \emptyset \mid (bs @ \varepsilon) \mid (bs @ x) \mid (bs @ ri_1 \oplus ri_2) \mid (bs @ ri_1 ri_2) \mid (bs @ ri^*).$$

In Rust. Figure 5.8 defines `ARegex` as the annotated counterpart of `Regex`. Every variant except `Phi` carries a leading `Vec<bool>` corresponding to the theoretical bit-list bs . The `Phi` variant has no annotation because it cannot contribute to a successful parse.

```
pub enum ARegex {
    Phi, // no annotation slot
    Eps(Vec<bool>), Lit(Vec<bool>, char),
    Alt(Vec<bool>, Box<ARegex>, Box<ARegex>),
    Seq(Vec<bool>, Box<ARegex>, Box<ARegex>),
    Star(Vec<bool>, Box<ARegex>),
}
```

Figure 5.8: The annotated-expression type.

Comparing this with Figure 2.1 shows the additional storage required for threading parse-tree information through the expression. Every variant except `Phi` therefore gains a leading `Vec<bool>` field. `Vec<bool>` mirrors the theoretical bs list directly, with one `bool` per bit, for a simple representation aligned with (5.8).

5.4.2 Annotated Derivatives

$ri \setminus_{bl}$ has the same case structure of (3.6), with bits threaded through instead of discarded:

$$\begin{aligned}
(bs @ ri_1 \oplus ri_2) \setminus_{bl} &= (bs @ (ri_1 \setminus_{bl}) \oplus (ri_2 \setminus_{bl})) & (B_+) \\
(bs @ ri_1 ri_2) \setminus_{bl} &= (bs @ (ri_1 \setminus_{bl}) ri_2) & (B_{seq}), \varepsilon \notin L(ri_1) \\
(bs @ ri_1 ri_2) \setminus_{bl} &= (bs @ (ri_1 \setminus_{bl}) ri_2) \oplus fuse(mkEpsBC_{ri_1}, ri_2 \setminus_{bl}) & (B_{seq}^n), \varepsilon \in L(ri_1) \\
(bs @ ri^*) \setminus_{bl} &= (bs @ fuse([0], ri \setminus_{bl}) ([] @ ri^*)) & (B_*).
\end{aligned} \tag{5.8}$$

The B_{seq}^n branch proceeds directly into ri_2 , so the bits produced by the empty match of ri_1 are fused into the derivative of ri_2 . This performs during the forward pass the reconstruction step that I_{seq}^R would otherwise perform during *inj*.

In Rust. Figure 5.9 implements the rules in (5.8). The base cases follow the ordinary derivative, while the nullable `Seq` and `Star` cases additionally use `mk_eps_bc` and `fuse` to preserve reconstruction information:

```
pub fn deriv_bc(ri: ARegex, l: char) -> ARegex {
  match ri {
    // Phi, Eps, Lit: unchanged from plain deriv (Figure~5.?)
    ARegex::Alt(bs, r1, r2) => {
      let d1 = deriv_bc(*r1, l);
      let d2 = deriv_bc(*r2, l);
      ARegex::Alt(bs, Box::new(d1), Box::new(d2))
    }
    ARegex::Seq(bs, r1, r2) => {
      if nullable_bc(&r1) {
        let eps_bits = mk_eps_bc(&r1);
        let d1 = deriv_bc(*r1, l);
        let d2 = deriv_bc(*r2.clone(), l);
        let left_branch = ARegex::Seq(vec![], Box::new(d1), r2);
        let right_branch = fuse(&eps_bits, d2);
        ARegex::Alt(bs, Box::new(left_branch), Box::new(right_branch))
      } else {
        let d1 = deriv_bc(*r1, l);
        ARegex::Seq(bs, Box::new(d1), r2)
      }
    }
    ARegex::Star(bs, r) => {
      let d = deriv_bc(*r.clone(), l);
      let fused = fuse(&[false], d);
      let new_star = ARegex::Star(vec![], r);
      ARegex::Seq(bs, Box::new(fused), Box::new(new_star))
    }
  }
}
```

Figure 5.9: `deriv_bc` (excerpt): the `Alt`, `Seq`, and `Star` arms implementing B_+ , $B_{\text{seq}}/B_{\text{seq}}^n$, and B_* ; the `Phi`/`Eps`/`Lit` base cases, unchanged from plain `deriv`, are omitted.

Each arm follows the same case structure as the plain `deriv` (Figure 3.3), with annotations threaded through the constructors and modified only where required by (5.8).

The `Alt` arm derives both operands and preserves the outer `bs`, since deriving does not change the choice already recorded for that alternation.

The `Seq` arm reflects B_{seq} and B_{seq}^n . In the non-nullable case, `r2` is moved directly into the rebuilt sequence. In the nullable case, `r2` is needed both unchanged in `left_branch` and for its derivative, so `r2.clone()` provides the second copy. Similarly, `r1` is borrowed by `nullable_bc` before being consumed.

The `Star` arm follows the same pattern. Its body is needed both for the derivative and the reconstructed star, so `r.clone()` provides the second copy. So both clones correspond to duplicated subexpressions in the corresponding derivative rules.

5.4.3 Internalizing Expressions

internalize recursively converts a plain expression r into an annotated expression ri . All nodes receive an empty bit-list except the two branches of an alternation, which are tagged with $0=Left$ and $1=Right$:

$$\begin{aligned}
internalize(\emptyset) &= \emptyset \\
internalize(\varepsilon) &= [] @ \varepsilon \\
internalize(x) &= [] @ x \\
internalize(r_1 + r_2) &= [] @ (fuse([0], internalize(r_1)) \oplus fuse([1], internalize(r_2))) \\
internalize(r_1 r_2) &= [] @ (internalize(r_1) internalize(r_2)) \\
internalize(r^*) &= [] @ internalize(r)^*.
\end{aligned} \tag{5.9}$$

The $[0]/[1]$ tags on the two sides of $r_1 + r_2$ follow the $0=Left/1=Right$ convention. Every other case introduces no new bit and uses only the empty list $[]$. The two calls to *fuse* in (5.9) are the only points in the construction where new bits are introduced.

In Rust. Figure 5.10 directly maps the constructors of **Regex** to their annotated counterparts, showing only the **Alt** case introduces non-empty annotations.

```

pub fn internalize(r: &Regex) -> ARegex {
  match r {
    Regex::Phi => ARegex::Phi,
    Regex::Eps => ARegex::Eps(vec! []),
    Regex::Lit(c) => ARegex::Lit(vec! [], *c),
    Regex::Alt(r1, r2) => {
      let ri1 = fuse(&[false], internalize(r1));
      let ri2 = fuse(&[true], internalize(r2));
      ARegex::Alt(vec! [], Box::new(ri1), Box::new(ri2))
    }
    Regex::Seq(r1, r2) => ARegex::Seq(vec! [],
      Box::new(internalize(r1)), Box::new(internalize(r2))),
    Regex::Star(r1) => ARegex::Star(vec! [], Box::new(internalize(r1))),
  }
}

```

Figure 5.10: *internalize*: convert a **Regex** into an **ARegex**.

The structure directly mirrors the six constructors of **Regex**, with one match arm for each constructor and each recursive call wrapped in the corresponding **ARegex** constructor.

The **Phi**, **Eps**, **Lit**, **Seq**, and **Star** cases use an empty bit-list, since they introduce no disambiguation choice at this stage. **Star** iteration choices are instead determined during derivative computation by B_* .

Only the **Alt** case calls *fuse* with a non-empty bit-list, tagging the left operand with $[false]$ and the right with $[true]$ before wrapping them in a fresh **Alt** with an empty outer bit-list. These are the only points where new bits are introduced; all later bits originate from these two tags and are propagated during derivation.

5.4.4 Fusing Bit-Lists

fuse modifies only the top node's bit-list and prepends *bs* to its existing list *bs'*. It does not descend into the children, leaving the recursively constructed subexpressions unchanged.

$$\begin{aligned}
fuse(bs, \emptyset) &= \emptyset \\
fuse(bs, (bs' @ \varepsilon)) &= (bs ++ bs') @ \varepsilon \\
fuse(bs, (bs' @ x)) &= (bs ++ bs') @ x \\
fuse(bs, (bs' @ ri_1 \oplus ri_2)) &= (bs ++ bs') @ (ri_1 \oplus ri_2) \\
fuse(bs, (bs' @ ri_1 ri_2)) &= (bs ++ bs') @ (ri_1 ri_2) \\
fuse(bs, (bs' @ ri^*)) &= (bs ++ bs') @ ri^*.
\end{aligned} \tag{5.10}$$

The $\emptyset$ case is unchanged, while all other cases prepend *bs*, which allows *fuse* to add branch information without inspecting the subexpression below the annotated node.

In Rust. Figure 5.11 implements this operation. The helper `combine` concatenates the bit-lists, while `fuse` preserves the structure below the top node. An empty `bs` returns the node unchanged, avoiding unnecessary rebuilding; otherwise only the top node is rebuilt. The `Phi` case is unchanged because it carries no annotation. The same operation later propagates bits from sub-derivations in B_{seq}^n and B_* .

```

pub fn fuse(bs: &[bool], ri: ARegex) -> ARegex {
    if bs.is_empty() { return ri; }
    match ri {
        ARegex::Phi => ARegex::Phi,
        ARegex::Eps(p) => ARegex::Eps(combine(bs, p)),
        ARegex::Lit(p, c) => ARegex::Lit(combine(bs, p), c),
        ARegex::Alt(p, r1, r2) => ARegex::Alt(combine(bs, p), r1, r2),
        ARegex::Seq(p, r1, r2) => ARegex::Seq(combine(bs, p), r1, r2),
        ARegex::Star(p, r) => ARegex::Star(combine(bs, p), r),
    }
}

fn combine(bs: &[bool], p: Vec<bool>) -> Vec<bool> {
    let mut new = bs.to_vec();
    new.extend(p);
    new
}

```

Figure 5.11: `fuse`: prepend a bit-list to the annotation of the top node.

5.4.5 Extracting Bits

mkEpsBC mirrors *mkEps* but returns bits, with the same left-preference guard at an alternation and a closing 1 at a star:

$$\begin{aligned}
mkEpsBC_{(bs @ \varepsilon)} &= bs, & mkEpsBC_{(bs @ ri^*)} &= bs ++ [1], \\
mkEpsBC_{(bs @ ri_1 ri_2)} &= bs ++ mkEpsBC_{ri_1} ++ mkEpsBC_{ri_2}, \\
mkEpsBC_{(bs @ ri_1 \oplus ri_2)} &= bs ++ \begin{cases} mkEpsBC_{ri_1} & \varepsilon \in L(ri_1) \\ mkEpsBC_{ri_2} & \text{otherwise.} \end{cases}
\end{aligned} \tag{5.11}$$

Where *mkEps* builds a `ParseTree` node by node, *mkEpsBC* only concatenates bit-lists. Each rule prepends the bits already present to those produced by the corresponding recursive call, without constructing a parse-tree node of its own.

In Rust. Compared with `mk_eps` (Figure 5.2), the case structure is unchanged; only the result differs, with `ParseTree` construction replaced by bit-list concatenation. The `Star` case does not recurse because a star is always nullable, so only the closing `true` is appended.

```

pub fn mk_eps_bc(ri: &ARegex) -> Vec<bool> {
    match ri {
        // Phi, Lit: unreachable, mk_eps_bc is called only once
        // nullable_bc(&ri) holds, and neither is ever nullable
        ARegex::Eps(bs) => bs.clone(),
        ARegex::Alt(bs, r1, r2) => {
            let mut result = bs.clone();
            if nullable_bc(r1) {
                result.extend(mk_eps_bc(r1));
            } else {
                result.extend(mk_eps_bc(r2));
            }
            result
        }
        ARegex::Seq(bs, r1, r2) => {
            let mut result = bs.clone();
            result.extend(mk_eps_bc(r1));
            result.extend(mk_eps_bc(r2));
            result
        }
        ARegex::Star(bs, _) => {
            let mut result = bs.clone();
            result.push(true);
            result
        }
    }
}

```

Figure 5.12: `mk_eps_bc` (excerpt): the `Eps`, `Alt`, `Seq`, and `Star` arms implement Equation (5.11)’s four cases directly; `Phi` and `Lit` only `panic!` on the (unreachable) precondition violation and are omitted.

5.4.6 Decoding Bits

$decode_r$ reconstructs a `ParseTree` from the bit-string produced by $mkEpsBC$. It traverses the original, unannotated r , since the annotated expression has been now reduced to bits. The recursive worker threads the remaining bit-list through the structure of r :

$$\begin{aligned}
decodeInner_\varepsilon(bs) &= (Empty, bs) \\
decodeInner_x(bs) &= (Char(x), bs) \\
decodeInner_{r_1+r_2}(0:bs) &= (Left(v), bs_1), & (v, bs_1) &= decodeInner_{r_1}(bs) \\
decodeInner_{r_1+r_2}(1:bs) &= (Right(v), bs_1), & (v, bs_1) &= decodeInner_{r_2}(bs) \\
decodeInner_{r_1r_2}(bs) &= (Pair(v_1, v_2), bs_2), & (v_1, bs_1) &= decodeInner_{r_1}(bs), \\
& & (v_2, bs_2) &= decodeInner_{r_2}(bs_1) \\
decodeInner_{r^*}(1:bs) &= (Star([], bs) \\
decodeInner_{r^*}(0:bs) &= (Star(v:vs, bs_2), & (v, bs_1) &= decodeInner_r(bs), \\
& & (Star(vs), bs_2) &= decodeInner_{r^*}(bs_1).
\end{aligned} \tag{5.12}$$

The outer function is $decode_r(bs) = v$ where $(v, []) = decodeInner_r(bs)$, and is undefined if bits remain. The *Alt* case reads the $0=Left/1=Right$ tags introduced by *internalize*. The *Star* case consumes one 0 per iteration until the trailing 1 inserted by $mkEpsBC$ terminates the sequence.

In Rust. Figure 5.13 implements (5.12) directly, with `decode_inner` matching its rules and `decode` providing the top-level wrapper.

Where `mk_eps_bc` creates bits by concatenation, `decode` consumes the flat bit-string from left to right while recursively traversing the tree-shaped r . The function `decode_inner` returns `(ParseTree, &[bool])`, producing the tree for the current subexpression together with the remaining bits. Rust’s slice patterns make the leading-bit cases direct, with `[false, rest @ ..]` and `[true, rest @ ..]` selecting an `Alt` branch or terminating a `Star`. `Seq` recurses once per factor, while `Star` uses a loop because its number of iterations is determined by the bits.

The outer `decode` asserts that no bits remain after decoding, enforcing the side condition in (5.12) and preventing leftover bits from being silently discarded.

```

pub fn decode(r: &Regex, bs: &[bool]) -> ParseTree {
    let (v, rest) = decode_inner(r, bs);
    assert!(rest.is_empty(),
        "decode: {} leftover bit(s) after decoding {:?}" , rest.len(), r);
    v
}

fn decode_inner<'a>(r: &Regex, bs: &'a [bool]) -> (ParseTree, &'a [bool]) {
    match r {
        Regex::Eps => (ParseTree::Empty, bs),
        Regex::Lit(c) => (ParseTree::Char(*c), bs),
        Regex::Alt(r1, r2) => match bs {
            [false, rest @ ..] => {
                let (v, remaining) = decode_inner(r1, rest);
                (ParseTree::Left(Box::new(v)), remaining)
            }
            [true, rest @ ..] => {
                let (v, remaining) = decode_inner(r2, rest);
                (ParseTree::Right(Box::new(v)), remaining)
            }
            [] => panic!("decode: unexpected end of bits at Alt"),
        },
        Regex::Seq(r1, r2) => {
            let (v1, after_r1) = decode_inner(r1, bs);
            let (v2, after_r2) = decode_inner(r2, after_r1);
            (ParseTree::Pair(Box::new(v1), Box::new(v2)), after_r2)
        }
        Regex::Star(r1) => {
            let mut iterations = Vec::new();
            let mut remaining = bs;
            loop {
                match remaining {
                    [true, rest @ ..] => { remaining = rest; break; }
                    [false, rest @ ..] => {
                        let (v, after) = decode_inner(r1, rest);
                        iterations.push(v);
                        remaining = after;
                    }
                    [] => panic!("decode: unexpected end of bits inside Star"),
                }
            }
            (ParseTree::Star(iterations), remaining)
        }
        // Phi: unreachable, decode never runs against a failed match
    }
}

```

Figure 5.13: `decode` and its recursive worker `decode_inner` (excerpt): every arm implements one of Equation (5.12)’s cases; the Φ precondition-panic arm is omitted.

5.4.7 Simplification

Unlike the direct construction the bit-coded method must simplify: Sulzmann and Lu show that a^* doubles in size at every derivative step without simplification. Simplification is safe here because the information required for reconstruction is encoded in the bits rather than in the unsimplified expression. The function *simp* is applied after every derivative step.

Sequence and star. The local simplification rules handle sequences and stars. A sequence propagates $\emptyset$, eliminates a leading ε while preserving its bits with *fuse*, and otherwise retains its structure. A star simplifies only its body:

$$\begin{aligned}
& \textit{simp}(\emptyset) = \emptyset \\
& \textit{simp}((bs @ \varepsilon)) = (bs @ \varepsilon) \\
& \textit{simp}((bs @ x)) = (bs @ x) \\
& \textit{simp}((bs @ ri_1 \text{ } ri_2)) = \begin{cases} \emptyset & \textit{isPhi}(\textit{simp}(ri_1)) \vee \textit{isPhi}(\textit{simp}(ri_2)) \\ \textit{fuse}(bs ++ bs_1, \textit{simp}(ri_2)) & \textit{simp}(ri_1) = (bs_1 @ \varepsilon) \\ (bs @ \textit{simp}(ri_1) \textit{simp}(ri_2)) & \text{otherwise} \end{cases} \\
& \textit{simp}((bs @ ri_1 \oplus ri_2)) = \textit{fuse}(bs, \textit{prune}(\textit{flatten}(ri_1, ri_2))) \\
& \textit{simp}((bs @ ri^*)) = (bs @ \textit{simp}(ri)^*)
\end{aligned} \tag{5.13}$$

The first case propagates the empty language through a sequence, while the second removes a leading ε and transfers its bits to the remaining expression. The *isPhi* test also recognizes expressions equivalent to $\emptyset$ beyond the literal $\emptyset$.

Alternation. Alternation requires additional restructuring. *flatten* collects nested alternatives that carry no bit-tag of their own into a single branch list. When descending through a tagged alternative, it first fuses its bits into both branches so that no recorded choice is lost. *prune* then simplifies each branch, removes branches equivalent to $\emptyset$, and removes duplicate annotated expressions while retaining the first occurrence. Finally, *collapse* rebuilds the remaining branches and applies the outer *bs* once:

$$\textit{simp}(bs @ (r_1 \oplus r_2)) = \textit{fuse}(bs, \textit{prune}(\textit{flatten}(r_1, r_2))). \tag{5.14}$$

The pruning step compares complete annotated expressions, including their bit-lists. Thus, branches with identical underlying expressions but different annotations remain distinct because they represent different reconstruction choices. If no branch survives, *collapse* produces $\emptyset$; with one survivor it returns that expression, and otherwise it rebuilds an alternation.

Because annotations are part of the expression, simplification does not merge branches solely on their underlying regex structure. Distinct bits can therefore keep branches separate even when their shapes coincide. This preserves the parse-tree choices encoded by the derivative, but also means that simplification does not guarantee bounded expression growth in every case.

5.4.8 Parser

Figure 5.14 implements the bit-coded driver. It internalizes r , applies `deriv_bc` and `simp` to each input character, checks nullability and, if successful, calls `mk_eps_bc` and `decode`.

```
pub fn parse_bitcoded_loop(input: &str, r: &Regex) -> Option<ParseTree> {
    let mut ri = internalize(r);
    for l in input.chars() { ri = simp(deriv_bc(ri, l)); }
    if !nullable_bc(&ri) { return None; }
    Some(decode(r, &mk_eps_bc(&ri)))
}
```

Figure 5.14: `parse_bitcoded_loop`: the bit-coded parser implementation.

Unlike the direct implementation, which has a growing `Vec<Regex>` for backward reconstruction, the bit-coded implementation needs only the current `ri`. The recursive entry point has the same three-step computation but differs only in how intermediate expressions are stored.

5.4.9 Worked Example

Consider the same input `ab` and regular expression $r_0 = (a \mid ab)(b \mid \varepsilon)$.

Internalize. Before consuming any input, `internalize` tags each alternative with `[0]` for the left branch and `[1]` for the right branch. These initial tags record the disambiguation choices that can later be propagated by `fuse`:

$$\text{internalize}(r_0) = ri_0 = []@([[]@([0]@a \oplus [1]@ab)) \cdot []@([0]@b \oplus [1]@ \varepsilon)). \quad (5.15)$$

Forward pass. The two input symbols are consumed by successive bit-threaded derivatives, with `simp` applied after each step. The first derivative uses B_{seq} . After consuming `a`, the first factor becomes nullable. The second derivative therefore uses B_{seq}^n , which propagates the bits of the empty match of the first factor into the derivative of the second factor:

$$\begin{aligned} ri_0 &\xrightarrow[B_{\text{seq}}]{a} ri_1 = []@([[]@([0]@ \varepsilon \oplus [1]@b) \cdot []@([0]@b \oplus [1]@ \varepsilon)) \\ ri_1 &\xrightarrow[B_{\text{seq}}^n]{b} ri_2 = []@([1]@([0]@b \oplus [1]@ \varepsilon) \oplus [0,0]@ \varepsilon). \end{aligned} \quad (5.16)$$

The `[1]` on the first surviving branch records the right alternative `ab` of the first factor. The `[0,0]` on the second branch results from fusing the bits of the empty match of the first factor into the derivative of $b + \varepsilon$. Thus, the information that would be recovered later by `inj` in the direct construction has already been propagated during the forward pass.

<pre> INTERNALIZE ----- ri0 = []@([]@([0]@'a' (+) [1]@('a'.'b')) . []@([0]@'b' (+) [1]@epsilon)) FORWARD PASS ----- Step 1: deriv_bc(ri0, 'a') rule: B_seq [nullable_bc(r1)=false] simp -> []@([1]@([0]@epsilon (+) [1]@'b')) . []@([0]@'b' (+) [1]@epsilon) nullable_bc(ri1) = true [OK] Step 2: deriv_bc(ri1, 'b') rule: B_seq^n [nullable_bc(r1)=true] simp -> []@([1]@([0]@'b' (+) [1]@epsilon) (+) [0,0]@epsilon) nullable_bc(ri2) = true [OK] </pre>	<pre> NULLABILITY CHECK ----- nullable_bc(ri2) = true -> proceed to mkEpsBC + decode MKEPSBC ----- mkEpsBC(ri2): left disjunct is nullable -> select Left inner alternative: b is not nullable epsilon is nullable -> select Right bits = [1,1] </pre>
--	---

Figure 5.15: Diagnostic trace of the bit-coded construction.

Extracting the bits. Both disjuncts of ri_2 are nullable, so $mkEpsBC$ applies EB_+^L and selects the left disjunct. Its outer annotation is empty, so no bit is emitted at that level. Inside the surviving alternative, b is not nullable while ε is, so EB_+^R selects the right branch and contributes its $[1]$ tag:

$$mkEpsBC(ri_2) \xrightarrow{EB_+^L} mkEpsBC([1]@([0]@b \oplus [1]@\varepsilon)) \xrightarrow{EB_+^R} [1] ++ [1] = [1, 1]. \quad (5.17)$$

Unlike $mkEps$, which constructs a partial parse tree, $mkEpsBC$ only extracts the committed choices as a flat bit-string. The tree structure is reconstructed by *decode*.

Result. The decoder reads $[1, 1]$ against the original, unannotated r_0 . The first bit selects the right branch of $a + ab$, producing the ab factor, while the second selects the right branch of $b + \varepsilon$:

$$decode_{r_0}([1, 1]) = (Right(a, b), Right()) = v_0. \quad (5.18)$$

Thus, the bit-coded construction produces the same POSIX parse tree as the direct construction, but without retaining the intermediate expressions r_0 , r_1 , and r_2 :

```

DECODE
-----
Bits:                [1,1]
Parse tree:          (Right (a, b), Right ())

RESULT
-----
Flattened:           "ab"
Current expression:  ri2

```

Figure 5.16: Final parse tree and validation of the bit-coded construction.

The resulting tree is validated by `flatten`:

$$\text{flatten}(v_0) = a \cdot b \cdot \varepsilon = \mathbf{ab}. \quad (5.19)$$

6 Partial-Derivative-Based Parser

Chapter 5 developed two parsers based on Brzowski's derivative. This chapter develops the corresponding two parsers using Antimirov's partial derivative from Section 3.3 in place of $r \backslash x$ throughout.

The replacement is local in notation but affects how the parsers are constructed. Both operators consume one character and return the residual expression that remains to be matched, and each operator is sufficient to drive a parser. However, $r \backslash x$ returns a single expression, whereas $pd(r, x)$ returns a *set*, and a set has no single top-level alternation whose left branch a rule can prefer. The POSIX choice performed by the two constructions of Chapter 5, in *mkEps* and in *mkEpsBC*, therefore has no corresponding site in the set representation. The choice is relocated, and its outcome differs.

Section 6.1 identifies the new site, and Sections ?? and 6.4.4 demonstrate the consequence on a small expression whose behaviour can be verified by direct inspection.

6.1 The Frontier Approach

Section 3.3 defined $pd(r, x)$ as a set of residual expressions, each representing a state of the implicit NFA for r after reading x . Parsing therefore pairs each surviving element with payload sufficient to reconstruct its parse tree: an injection closure in this project's extension (Section 6.2) or a bit string in the reference-based construction (Section 6.4).

The algorithm maintains a *frontier* of (**residual**, **extra**) pairs, one per live *strand*, and replaces it after each input character with the successors produced by pd . At end of input, it selects the first strand with a nullable residual. Figure 6.1 illustrates this process using `pderiv_bc`'s bits as payload; Section 6.2 shows the frontier with injection closures.

Both strands in the last column are nullable, yet only the first is inspected. Neither construction defines a preference; list order determines which is considered first. This is deliberate: parse trees are complete only at end of input and cannot be compared during pruning. Sections 6.2 and 6.4 develop complete parsers from this loop.

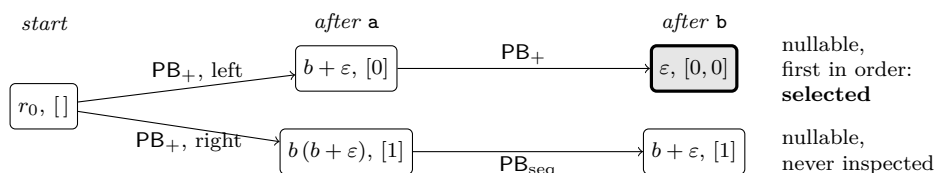


Figure 6.1: Frontier for $r_0 = (a + ab)(b + \varepsilon)$ on "ab", using `pderiv_bc`'s bits as per-strand payload. Both final strands are nullable; the first in list order is selected.

6.2 The Standard Partial-Derivative-Based Parser

6.3 The Bit-Coded Approach

`pderiv_std`'s frontier carries a chain of heap-allocated closures, one per surviving derivative step, applied only once, on the winning strand, at the very end. `pderiv_bc` replaces that chain with the same flat `Vec<bool>` Chapter 5's `deriv_bc` used, following Sulzmann's teaching-page Haskell transcript [Sulzmann, 2026] directly rather than building a parallel construction from scratch. Figure 6.1 (Section 6.1) already draws this representation in full, so there is no second frontier diagram to redraw here: reading it again now, the payload on every strand is a `Vec<bool>`, not an ι .

Comparison with the standard construction.

	<code>pderiv_std</code>	<code>pderiv_bc</code>
Per-strand payload	<code>Rc<dyn Fn></code> closure chain	<code>Vec<bool></code>
Allocation per step	one heap closure per successor	one <code>Vec</code> append
Residual constructor	plain <code>Regex::seq</code> only	<code>smart_seq</code> where safe
Followed reference	none; this thesis's own	<code>pDerivBC</code> directly

Both constructions deduplicate identically, `nub_tree` and `nub2` apply the same keep-the-first rule to the same residual equality, so the difference in this table is entirely about what rides alongside the residual, never about which residual survives. The next section develops `pderiv_bc`'s rules formally and sets its own per-case table next to Section ??'s, now that both constructions are available to compare.

6.4 The Bit-Coded Partial-Derivative-Based Parser

Sulzmann's teaching-page Haskell transcript gives a bit-coded partial-derivative construction, `pDerivBC`, that this thesis's `pderiv_bc` follows directly [Sulzmann, 2026]. It extends `pd` with per-strand bits the same way Section ??'s $\backslash_b$ extends the Brzozowski derivative, with one visible difference: the bits stay *external* to the expression tree, as a plain list carried alongside a plain, un-annotated r , instead of being folded into an `ARegex`-shaped tree. This construction's correctness criterion is literal agreement with `pDerivBC`, not an independently derived ideal form.

6.4.1 The Bit-Coded Partial Derivative

Writing D_r for `pDerivBC( $x$ ,  $r$ )` throughout, to keep the set comprehensions readable, the base and non-sequence cases are

$$\begin{aligned}
D_\varepsilon &= \{\} & D_\emptyset &= \{\} & (\text{PB}_\varepsilon), (\text{PB}_\emptyset) \\
D_y &= \begin{cases} \{(\varepsilon, [])\} & x = y \\ \{\} & x \neq y \end{cases} & & (\text{PB}_{\text{lit}}) \\
D_{r+s} &= \text{nub2}\left(\{(r', 0:bs) \mid (r', bs) \in D_r\} \right. & & (6.1) \\
&\quad \left. \cup \{(s', 1:bs) \mid (s', bs) \in D_s\}\right) & & (\text{PB}_+) \\
D_{r^*} &= \text{nub2}\left(\{(smartC(r', r^*), 0:bs) \mid (r', bs) \in D_r\}\right) & & (\text{PB}_*)
\end{aligned}$$

and the sequence case splits on the nullability of the left factor exactly as Equation (3.6) does:

$$\begin{aligned}
D_{rs} &= \{(smartC(r', s), bs) \mid (r', bs) \in D_r\} & (\text{PB}_{\text{seq}}), n(r) = \text{false} \\
D_{rs} &= \text{nub2}\left(\{(r's, bs) \mid (r', bs) \in D_r\} \right. & (6.2) \\
&\quad \left. \cup \{(s', mkEpsBC_r ++ bs) \mid (s', bs) \in D_s\}\right) & (\text{PB}_{\text{seq}}^n), n(r) = \text{true}
\end{aligned}$$

The names extend Section 3.3's P scheme, one PB rule per P rule, and the derivation below cites them with the binding of r and s given at each application. Here $mkEpsBC_r$ is the plain- r analogue of Section ??'s $mkEpsBC$, identical in structure but walking an unannotated tree; $smartC$ is the smart concatenation constructor of Section 3.3, collapsing $\varepsilon \cdot s$ to s ; and nub2 deduplicates a list of (residual, bits) pairs by comparing the residual alone, keeping the first occurrence.

Two details of Equation (6.2) deserve attention. PB_{seq}^n 's second set is where a strand inherits $mkEpsBC_r$, the bits r would have contributed by matching empty, which is the same move B_{seq}^n made with fuse in Chapter 5 and the same one I_{seq}^R made with $mkEps$. And the two sequence cases build their residuals differently: PB_{seq} uses $smartC$ while PB_{seq}^n builds its first strand with a plain $r's$. That asymmetry is present in the reference itself and is preserved here and not "corrected", since correctness of this construction is judged by literal agreement with that reference (Chapter 8) and not against an independently derived ideal form. Section ?? met the same question from the other side, where the choice between the two constructors was not free.

In Rust. `pderiv_bc` transcribes $pDerivBC$ case for case. Each D_r becomes a vector of (Regex, Vec<bool>) pairs and not a set, because order is what encodes priority here, and a `HashSet` would discard exactly the information selection depends on. Deduplication is factored into its own function, Figure 6.2, used by every case that can produce more than one strand.

```

fn nub2(pairs: Vec<(Regex, Vec<bool>>>) -> Vec<(Regex, Vec<bool>>>) {
    let mut seen: HashSet<Regex> = HashSet::with_capacity(pairs.len());
    let mut out = Vec::with_capacity(pairs.len());
    for (r, bs) in pairs {
        if seen.insert(r.clone()) {
            out.push((r, bs));
        }
    }
    out
}

```

Figure 6.2: `nub2`: stable deduplication on the residual alone. `HashSet::insert` returns `true` the first time a value is seen and `false` on every repeat, so the loop keeps a pair exactly when its residual has not been seen before, in the order the input `Vec` presents them.

This is the one place in the codebase where `Regex`'s derived `Hash` implementation (Section 2.3) is load-bearing: elsewhere equality alone would do, but a hash set is what keeps `nub2` linear in the number of strands instead of quadratic.

Figure 6.3 gives the two arms that produce more than one strand and tag them as they go; `Star` is left out, since it tags its one strand with a single leading `false` the same way `Alt`'s left arm does, and otherwise matches Figure 5.9's star arm in Chapter 5.

```

Regex::Alt(r1, r2) => {
  let mut out: Vec<(Regex, Vec<bool>)> = pderiv_bc(r1, x)
    .into_iter()
    .map(|(r1_, mut bs)| {
      let mut b = vec![false];
      b.append(&mut bs);
      (r1_, b)
    })
    .collect();
  out.extend(pderiv_bc(r2, x).into_iter().map(|(r2_, mut bs)| {
    let mut b = vec![true];
    b.append(&mut bs);
    (r2_, b)
  }));
  nub2(out)
}

Regex::Seq(r1, r2) => {
  if nullable(r1) {
    let mut out: Vec<(Regex, Vec<bool>)> = pderiv_bc(r1, x)
      .into_iter()
      .map(|(r1_, bs)| (Regex::seq(r1_, *r2.clone()), bs))
      .collect();
    let eps_bits = mk_eps_bits(r1);
    out.extend(pderiv_bc(r2, x).into_iter().map(|(r2_, bs)| {
      let mut b = eps_bits.clone();
      b.extend(bs);
      (r2_, b)
    }));
    nub2(out)
  } else {
    pderiv_bc(r1, x)
      .into_iter()
      .map(|(r1_, bs)| (smart_seq(r1_, r2), bs))
      .collect()
  }
}

```

Figure 6.3: The alternation and sequence arms of `pderiv_bc`, transcribing PB_+ , PB_{seq} and PB_{seq}^n .

`Alt` prepends its tag with `vec![false]/vec![true]` and `append`, which is $0:bs$ and $1:bs$ read as a `Vec` operation, then deduplicates the combined vector once at the end, matching `nub2`'s position on the outside of the union in Equation (6.1).

`Seq` is the one place the two constructors genuinely differ, and the code keeps that difference instead of smoothing it away. The nullable branch's first strand is built with plain `Regex::seq(r1_, *r2.clone())`, matching PB_{seq}^n 's plain r 's; its second strand reuses `mk_eps_bits(r1)` exactly as Section 6.4.1 described. The non-nullable branch, reached only when `nullable(r1)` is false, calls `smart_seq(r1_, r2)` instead, matching PB_{seq} . A reader who only skims the two branches could mistake this for an inconsistency; it is the

literal transcription of the asymmetry Equation (6.2) already has.

6.4.2 Comparing the Two Constructions

Section ??’s table and this section’s rules are the same case split read twice, once attaching a closure and once a bit:

Case	<code>pderiv_bc</code> attaches	<code>pderiv_tree</code> attaches
$\varepsilon, \emptyset$	no strands	no strands
x (matching)	<code>[]</code>	<code> _ Char(x)</code>
$r + s$, left	<code>0:bs</code>	<code> v Left(inj1(v))</code>
$r + s$, right	<code>1:bs</code>	<code> v Right(inj2(v))</code>
rs , continue r	<code>bs</code>	<code> Pair(v1,v2) Pair(inj1(v1), v2)</code>
rs , jump to s	<code>mkEpsBC_r ++ bs</code>	<code> v2 Pair(mk_eps(r), inj2(v2))</code>
r^*	<code>0:bs</code>	<code> Pair(v1,vs) Star(inj1(v1) : vs)</code>

Every row is the same decision, expressed once as a bit and once as a constructor. The correspondence is exact enough that the "jump to s " row calls the shared `mk_eps` at precisely the point `pderiv_bc` calls `mk_eps_bits`, for the reason l_{seq}^R gave in Chapter 5. `nub_tree` is called in exactly the positions `nub2` is in Figure 6.3, deduplicating on the residual alone and keeping the first occurrence, so the priority order is preserved identically between the two constructions.

Where `nub2` is and is not applied. `nub2` appears inside $pDerivBC$, once per case that can produce more than one strand, and nowhere else. In particular the frontier-driving loop concatenates the per-strand results into the next frontier without deduplicating across them, so the guarantee `nub2` provides is local to a single strand’s expansion and does not extend to the frontier as a whole. Two strands that were distinct at step i can converge on the same residual at step $i + 1$ and both survive, exactly the same effect `nub_tree` permits in `pderiv_std`, since both apply the same keep-the-first rule only within a single strand’s expansion.

6.4.3 Parser

`step_frontier` and `select_bits` implement the loop of Section 6.1. Figure 6.4 gives both, plus the entry point that calls them.

```

pub(super) fn step_frontier(frontier: &[(Regex, Vec<bool>)]), x: char)
    -> Vec<(Regex, Vec<bool>>>
{
    let mut out = Vec::new();
    for (r, bits) in frontier {
        for (r_next, extra) in pderiv_bc(r, x) {
            let mut combined = bits.clone();
            combined.extend(extra);
            out.push((r_next, combined));
        }
    }
    out
}

pub(super) fn select_bits(frontier: &[(Regex, Vec<bool>)]))
    -> Option<Vec<bool>>
{
    for (r, bits) in frontier {
        if nullable(r) {
            let mut result = bits.clone();
            result.extend(mk_eps_bits(r));
            return Some(result);
        }
    }
    None
}

pub fn parse_pderiv_bc(input: &str, r: &Regex) -> Option<ParseTree> {
    let mut frontier = vec![(r.clone(), Vec::new())];
    for c in input.chars() {
        frontier = step_frontier(&frontier, c);
        if frontier.is_empty() {
            return None;
        }
    }
    let bits = select_bits(&frontier)?;
    Some(decode(r, &bits))
}

```

Figure 6.4: The frontier loop and its entry point. `step_frontier` replaces every (`residual`, `bits`) pair with the union of `pderiv_bc` applied to it, concatenating each result’s bits onto that strand’s accumulated prefix. `select_bits` scans in list order and returns the first nullable strand’s bits ++ `mk_eps_bits(residual)`.

`parse_pderiv_bc` drives the loop and passes the winning bit-string, together with the *original* `Regex` and never the derived residual, to `decode` (Section 5.4.6). That is the same shared decoder `deriv_bc` uses, since both constructions arrive at the identical (`Regex`, `bits`) shape by the time decoding is needed, whatever route each took to get there. The early return on an empty frontier is the same short-circuit Figure 3.8’s matcher uses, now

guarding a $\text{Vec}(\text{Regex}, \text{Vec}(\text{bool}))$ instead of a $\text{HashSet}(\text{Regex})$.

6.4.4 Worked Example

Run the same r_0 against `ab` through this construction, checked against `cargo run -parse "(a|ab)(b|)" "ab" -parser pderiv_bc -diag 3` (Appendix A.4), so it can be read side by side with Section ??’s run of the standard construction on the identical input.

Forward pass. The frontier starts as one strand, $(r_0, [])$. The first character binds $r = a + ab$ and $s = b + \varepsilon$ at the outer node. That r is not nullable, so PB_{seq} applies and s contributes no strand; the strands that do appear come from PB_+ one level down:

$$\begin{aligned}
 D_{a+ab} &= \{(\varepsilon, [0])\} \cup \{(b, [1])\} && \text{PB}_+, r = a, s = ab \\
 D_{r_0} &= \{(\text{smartC}(\varepsilon, b + \varepsilon), [0]), (\text{smartC}(b, b + \varepsilon), [1])\} && \text{PB}_{\text{seq}}, n(a + ab) = \text{false} \\
 &= \{(b + \varepsilon, [0]), (b(b + \varepsilon), [1])\} && \text{SC twice}
 \end{aligned} \tag{6.3}$$

The two applications of **SC** take different branches, which is why the two strands look so different from Section ??’s at the same step. Against the first, $r' = \varepsilon$, so **SC** absorbs it and leaves $b + \varepsilon$ alone; against the second, $r' = b \neq \varepsilon$, so it keeps the concatenation. The trace agrees:

```

Step 1 (char: 'a', position 1)
  rule: pDerivBC('a', r1.r2) [nullable(r1)=false]
        = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
frontier after (2 residual(s)):
  [0] Alt(Lit('b'), Eps)    bits = [0]
  [1] Seq(Lit('b'), Alt(Lit('b'), Eps)) bits = [1]

```

Both bits were emitted by PB_+ one level down, and PB_{seq} carried them outward unchanged, which is why neither strand’s bits grew at this step. Where the plain bit-coded construction of Chapter 5 kept the two alternatives inside one expression, joined by $\oplus$, they are now two separate objects that will be advanced independently.

The second character steps *both* strands, and each is matched against the rule set on its own, so two different rules fire on the same character:

$$\begin{aligned}
 D_{b+\varepsilon} &= \{(\varepsilon, [0])\} \cup \{\} && \text{PB}_+, r = b, s = \varepsilon; \text{PB}_\varepsilon \\
 D_{b(b+\varepsilon)} &= \{(\text{smartC}(\varepsilon, b + \varepsilon), [0])\} = \{(b + \varepsilon, [0])\} && \text{PB}_{\text{seq}}, n(b) = \text{false}
 \end{aligned} \tag{6.4}$$

Strand [0] is an alternation whose left disjunct contributes ε tagged 0 and whose right contributes nothing, since PB_ε returns the empty set. Strand [1] is a concatenation whose left factor is a bare literal, so PB_{seq} applies and only that factor is derived. Each strand’s accumulated bits are its own prefix followed by whatever the step produced, so [0] gains a 0 and reaches [0,0] while [1] gains nothing and stays at [1]:

```

Step 2 (char: 'b', position 2)
  from [0]:

```

```

rule: pDerivBC('b', r1+r2) = tag(0, pDerivBC('b',r1)) ++
      tag(1, pDerivBC('b',r2)), nub2
-> (Eps, [0])
from [1]:
rule: pDerivBC('b', r1.r2) [nullable(r1)=false]
      = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
-> (Alt(Lit('b'), Eps), [])
frontier after (2 residual(s)):
[0] Eps    bits = [00]
[1] Alt(Lit('b'), Eps)    bits = [1]

```

Selection and result. Selection scans this final frontier in list order and takes the first nullable strand. Strand [0], $(\varepsilon, [0, 0])$, is checked and accepted before strand [1] is inspected, although [1] is nullable too:

$$[0]\text{'s bits} ++ mkEpsBC(\varepsilon) = [0, 0] ++ [] = [0, 0], \quad decode_{r_0}([0, 0]) = (Left\ a, \ Left\ b). \quad (6.5)$$

selected residual's accumulated bits ++ mkEpsBC(residual) = [00]

```

decode((a|ab)(b|), [00]):
= (Left a, Left b)

```

RESULT

```

Bits:                [00]
Parse tree:          (Left a, Left b)
Flattened:            "ab"

```

This is the identical tree Section ?? reached by composing injection closures, reached here by reading bits off the winning strand instead. Two differences from that section's trace are visible, and both were predicted by the smart-constructor argument of Section ?. First, the residuals themselves: where this construction lists `Alt(Lit('b'), Eps)` after `a`, Section ?? lists `Seq(Eps, Alt(Lit('b'), Eps))`, the ε node *smartC* discards here and the other construction cannot afford to. Second, and sharper: because the residuals differ, the two constructions do not even take the same rules at step two. This construction's strand [0] is an `Alt` and fires PB_+ ; the standard construction's is a `Seq` whose left factor is the retained `Eps`, so it fires the nullable sequence rule instead. The two constructions are therefore not running over the same residuals, and not applying the same rules to them, and they still select the same tree. Because deduplication runs on residual syntax in both, they can in principle prune at different points as well, which makes their agreement on every tested case a stronger check than it would be if they shared a residual representation.

7 External To Internal Regex

The examples presented thus far have been expressed directly in terms of **Regex**, the six-constructor core algebra introduced in Section 2.3. This is the representation consumed by **deriv** and **pderiv**. External regular-expression patterns, however, are provided as strings using a PCRE-style syntax. For example, a pattern such as "[a-c]+\d{2}" must first be converted into the internal representation before it can be processed by the parsers.

The frontend in `src/frontend/` provides this conversion. It first parses the external pattern into a surface-syntax AST and then translates that AST into the **Regex** representation used by the parsers in Chapters 5 and 6. Its design is based on the `Text.Regex.PDeriv` implementation in `luzhuomi/regex-pderiv`, in particular the `ExtPattern`, `Parse`, and `Translate` components used in Kenny Lu's implementation of the partial-derivative approach of Sulzmann and Lu [2012]. This implementation provides a separate reference for the treatment of external syntax. The construction in Sulzmann and Lu [2014], by contrast, operates directly on the six-constructor **Regex** algebra and does not define a PCRE-oriented frontend.

The real-world corpus described in Section 8.2 depends on this frontend to convert Suriata, SpamAssassin, and RegexLib rule patterns into **Regex** values. This conversion includes parsing the surface syntax, desugaring constructs that are not represented by **Regex**, and applying the substring-search padding scheme.

7.1 Two ASTs

The frontend separates surface syntax from the internal regular-expression algebra by using two abstract syntax trees connected by a single translation step:

$$\text{pattern string} \xrightarrow{\text{parse_ext_pattern}} \text{ExtPat} \xrightarrow{\text{translate}} \text{Regex}$$

ExtPat (`ext_pattern.rs`) represents the external surface syntax. It includes constructs such as marking and non-marking groups, bounded repetitions, character-class shorthands such as `\d`, `\w`, and `\s`, anchors, and the **greedy**: `bool` field associated with quantifiers. These constructs remain explicit in **ExtPat** and are not desugared during parsing.

Regex, introduced in Section 2.3, is the internal six-constructor representation `Phi | Eps | Lit | Seq | Alt | Star`. It has no representation for groups, repetition bounds, anchors, or quantifier laziness. Constructs expressible in **ExtPat** but absent from **Regex** are therefore desugared by `translate` (Section 7.3). Anchors are handled separately:

`\^` and `$` are translated to `Eps`, while their effect on substring search is handled by the frontend entry points described in Section 7.4.

The explicit intermediate representation also isolates parsing from translation. The unit tests in `translate.rs`, for example, can construct `ExtPat` values directly and verify their translated `Regex` representations without depending on the recursive-descent parser in `parse.rs`. This separation is analogous to the distinction between parse-tree construction and operations such as `flatten` and `decode` discussed in Section 4.2: each stage has a distinct input and output representation and can therefore be tested independently.

7.2 Parsing Surface Syntax

The parser in `frontend/parse.rs` is a recursive-descent parser operating on a `Chars` iterator. Its structure follows the grammar used by `Text.Regex.PDeriv.Parse`. The reference functions `p_ere`, `p_branch`, `p_exp`, `p_atom`, `p_group`, `p_charclass`, `p_enum`, and `p_bound` correspond to `parse_or`, `parse_concat`, `parse_postfixed`, `parse_atom`, `parse_group`, `parse_charclass`, `parse_class_enum`, and `parse_bound`. These functions implement the respective levels of the surface grammar.

The implementation differs from the reference parser in several respects.

- **Class shorthands are interpreted as character classes.** The reference implementation’s `Translate.hs` treats a bare `EEscape c` as the literal character `c`. This frontend instead interprets `d`, `D`, `w`, `W`, `s`, and `S` as character-class shorthands (Section 7.3). This interpretation corresponds to their use in the external patterns processed by the project.
- **Backreferences and lookahead are rejected.** Constructs such as `\1–\9` and lookahead assertions result in parse errors. Backreferences and lookahead are not represented by the regular-expression algebra used by the derivative constructions. Rejecting them explicitly prevents constructs outside the supported formal model from being silently misinterpreted. This is important for corpus extraction described in Section 8.2, where unsupported constructs must be identified explicitly.
- **Named groups are accepted.** Both `(?<name>...)` and `(?P<name>...)` are parsed and subsequently translated to their unnamed group contents. Group names have no representation in `Regex` and therefore have no effect after translation. Supporting the syntax nevertheless permits patterns containing named groups to be processed without introducing capture-group semantics into the internal representation.
- **Hexadecimal escapes are accepted.** The parser additionally accepts `\xHH` hexadecimal character escapes, which are not provided by the reference frontend.

The function `parse_bound` also recognizes lazy-quantifier markers, such as `a*?`, `a+?`, and `a{2,4}?`. The marker is stored in the `greedy: bool` field of `ExtPat`, although it does not affect the subsequent translation, as described in Section 7.3.

The precedence of the surface syntax follows the reference grammar. Alternation has the lowest precedence, followed by concatenation and then postfix quantifiers, with each quantifier applying to a single atom. At the atom level, the parser handles literals, escapes, character classes, groups, and anchors.

7.3 Translating ExtPat into Regex

The function `translate` in `frontend/translate.rs` is defined by structural recursion over `ExtPat`. Each constructor has a corresponding match arm that either maps directly to a `Regex` constructor or explicitly desugars the surface construct:

ExtPat	Regex
<code>Empty, Eps</code>	<code>Eps</code>
<code>Never</code>	<code>Phi</code>
<code>Char(c)</code>	<code>Lit(c)</code>
<code>Group/GroupNonMarking(r)</code>	<code>translate(r)</code>
<code>Or([r1, ..., rn])</code>	left-folded <code>Alt</code> chain
<code>Concat([r1, ..., rn])</code>	left-folded <code>Seq</code> chain
<code>Opt(r)</code>	<code>Alt(translate(r), Eps)</code>
<code>Plus(r)</code>	<code>Seq(translate(r), Star(translate(r)))</code>
<code>Star(r)</code>	<code>Star(translate(r))</code>
<code>Carat, Dollar</code>	<code>Eps</code>
<code>Dot, Any(cs), NoneOf(cs)</code>	sorted, deduplicated <code>Alt</code> of <code>Lit</code>
<code>Bound(r, lo, hi)</code>	unrolled exact/optional repetition

The translation therefore removes all surface constructs that are not present in the core algebra while preserving the language represented by the supported constructs.

Character classes are represented by alternation. `Any`, `NoneOf`, `Dot`, and class shorthands are translated via `alt_of_chars`, which sorts and deduplicates the character set into a left-folded `Alt` chain. The file `alphabet.rs` provides 98 characters (printable ASCII, tab, newline, carriage return). The functions `digit_chars`, `word_chars`, and `space_chars` define sets for `\d`, `\w`, and `\s`, with negated forms via `complement`.

The `Regex` representation contains no dedicated character-class constructor. Consequently, every character class is reduced to the six constructors of the core algebra. For example, a `\d` shorthand is represented as an alternation over its ten characters, which the derivative implementations subsequently process through the ordinary `Alt` structure.

Bounds are desugared by explicit repetition. The function `desugar_bound` constructs the exact lower-bound portion using `repeat_exact`. This function produces a left-folded `Seq` chain and returns `Eps` when `lo == 0`. For an unbounded upper limit, the remaining repetitions are represented by a `Star`. Thus, `r{2,}` becomes two explicit copies followed by `Star(r)`. For a finite upper bound, `hi - lo` additional copies are represented using nested optional expressions constructed by `optional_extra`.

Bounded repetition can substantially increase the size of the internal expression. A short source pattern such as `r{1586,}` would otherwise produce 1586 nested `Seq` nodes before the trailing `Star` is introduced. The frontend therefore imposes `MAX_BOUND = 1000`. This limit is specific to the implementation and is not inherited from the reference parser. Bounds exceeding the limit result in a descriptive translation error rather than the construction of an excessively large `Regex`.

Greedy and lazy quantifiers are parsed but not distinguished. The translation of `Opt`, `Plus`, `Star`, and `Bound` does not inspect the `greedy` flag recorded by `parse_bound`. The internal `Regex` representation has no per-quantifier notion of greediness or laziness. Disambiguation is instead determined by the parser as a whole, with the derivative-based and partial-derivative-based constructions implementing their respective policies, as described in Section 4.3.

Consequently, `a*` and `a*?` translate to identical `Regex` values. A pattern containing both greedy and lazy quantifiers can therefore be parsed successfully, but the distinction between the two quantifier forms is not preserved by translation. The frontend consequently accepts a broader surface syntax than can be distinguished by the internal representation.

7.4 Substring Search

The parsers in Chapters 5 and 6 perform full-string matching: after the input has been consumed, the resulting expression is tested for nullability. They do not provide a separate operation for determining whether a pattern occurs as a substring of a longer input. The derivative constructions of Sulzmann and Lu [2014] and the corresponding teaching material likewise define matching over the complete input.

The external patterns used by Suricata and SpamAssassin generally have search semantics, where a rule is triggered when its pattern occurs within a larger payload. The frontend therefore implements substring search before the resulting `Regex` is passed to a parser. The translated pattern is surrounded by `wildcard_run()`, a `Star` over a 98-character alternation representing an arbitrary sequence of input characters. Padding is added only on sides that are not explicitly anchored:

$$\begin{aligned} \backslash^{\wedge}\text{abc} &\rightarrow \text{abc} \cdot \Sigma^* && \text{(leading anchor: pad the end only)} \\ \text{abc}\$ &\rightarrow \Sigma^* \cdot \text{abc} && \text{(trailing anchor: pad the start only)} \\ \text{abc} &\rightarrow \Sigma^* \cdot \text{abc} \cdot \Sigma^* && \text{(no anchor: pad both sides)} \\ \backslash^{\wedge}\text{abc}\$ &\rightarrow \text{abc} && \text{(fully anchored: no padding)} \end{aligned}$$

The functions `starts_with_carat` and `ends_with_dollar`, defined in `ext_pattern.rs`, determine whether padding is required. They inspect the structure of `Concat` and `Group` nodes to identify leading `\^` and trailing `$` anchors, including anchors nested inside groups. The anchors themselves are subsequently translated to `Eps`, since their effect has already been accounted for when determining the padding.

This behaviour applies to the search-oriented entry points `parse_dataset_pattern` and `parse_pcre_rule`. The function `parse_pattern`, used for full-string examples throughout the thesis, does not introduce padding and retains ordinary full-string matching semantics.

`parse_pcre_rule` additionally unwraps PCRE delimiters and case-folds on the `i` flag. Suricata and SpamAssassin rule text is not a bare pattern: it carries its own `/PATTERN/FLAGS` wrapper, the same convention PCRE tools use to separate the pattern from modifiers such as case-insensitivity. `parse_pcre_rule` strips this wrapper with `strip_pcre_delimiters` before parsing, and, if the `i` flag is present, applies `case_fold`

to the resulting `ExtPat` before translation. `case_fold` rewrites every literal character into an `Any` of both cases, so `a` becomes `Any(['a', 'A'])`; the fold happens at the `ExtPat` level rather than at match time because `Regex` itself has no case-insensitivity concept for a parser to consult. `parse_dataset_pattern` has no such flag to inspect and is always case-sensitive. `parse_pattern` passes any input through unchanged if it is not `/`-delimited, so a bare pattern with no wrapper is accepted by either search entry point exactly as written.

Search finds *a* match, not necessarily the first one. Padding turns "found somewhere in this input" into an ordinary full-match question over a larger expression, and that substitution is not free: it inherits whichever iteration count a `Star` prefers under the policy in force, and both policies this thesis implements prefer *more* iterations of the leading Σ^* when the rest of the input can still be consumed, exactly as either would prefer more iterations of any other `Star`.

At every character, the leading Σ^* faces the same fork any nullable first factor of a `Seq` produces: keep deriving into the `Star` itself (treat this character as filler), or also derive into what follows starting here (try to begin the core pattern at this position). Both survive as one `Alt`; whichever survives to the end is read off by the disambiguation policy in force. The following uses a reduced three-letter alphabet $\{a, b, c\}$ in place of the real 98-character Σ , purely so the resulting parse tree stays short enough to display; the mechanism is identical either way. With $r = \Sigma^* ab \Sigma^*$ against "cab`c`", which contains `ab` once, at position 1:

$$v_1 = \left(\left((Star[Right(c)], a), b \right), Star[Right(c)] \right) \quad (7.1)$$

the leading Σ^* takes one iteration (`c`), the literal `ab` matches, and the trailing Σ^* takes one iteration (`c`) — exactly where `ab` sits. Against "abcab", which contains `ab` *twice*, at position 0 and position 3:

$$v_2 = \left(\left((Star[Left(Left(a)), Left(Right(b)), Right(c)], a), b \right), Star[] \right) \quad (7.2)$$

the leading Σ^* instead runs *three* iterations (`a`, `b`, `c`) before yielding to the literal `ab`, and the trailing Σ^* is empty: at position 0, consuming `a` as filler and finding `ab` later is a complete, valid parse, so the policy takes it, and (7.2) reports the *second* occurrence, not the first.

8 Evaluation

Chapters 5 and 6 developed four parser combinations, `deriv_std` (recursive and loop variants), `deriv_bc`, `pderiv_std` and `pderiv_bc`, each claimed to compute either the POSIX or the leftmost-first parse tree, by four structurally different constructions. This chapter evaluates those four parsers against each other, first for correctness and then for performance, on two kinds of input: 1. Small, constructed patterns taken from the source paper to isolate one algorithmic property at a time; 2. A corpus of real-world regular expressions assembled from three sources: Snort/Suricata, SpamAssassin and RegexLib, that Mamouras et al.’s disambiguation-robustness study [Mamouras et al., 2024] uses.

That paper studies exactly the POSIX-versus-Greedy question this thesis studies, via real intrusion-detection signatures, and finds concrete real-world patterns where the two policies visibly diverge. It provides no public dataset artifact of its own, so Section 8.2 describes how the corpus was independently assembled and prepared.

Section 8.1 checks the four parsers, and the three Boolean matchers of Chapter 3, against each other and against the paper’s own worked examples. Section 8.2 describes both input sources: synthetic patterns and the real-world corpus. Section 8.3 measures where the four parsers’ costs diverge on each and the final Section 8.4 draws the two together.

8.1 Parser Agreement

Paper A1 Example:. $r_1 = (a + ab)(b + \varepsilon)$ and $w = ab$,

```
$ cargo run -- parse "(a|ab)(b|)" "ab" --parser all
Parser      | Policy      | Result
-----+-----+-----
DERIV_REC   | POSIX       | (Right (a, b), Right ()) -> "ab"
DERIV_LOOP  | POSIX       | (Right (a, b), Right ()) -> "ab"
DERIV_BC    | POSIX       | (Right (a, b), Right ()) -> "ab"
PDERIV_STD  | NON-POSIX   | (Left a, Left b) -> "ab"
PDERIV_BC   | NON-POSIX   | (Left a, Left b) -> "ab"

POSIX parsers agree
PDERIV parsers agree
NON-POSIX agrees with POSIX on membership
```

Figure 8.1: Comparison of the parser results for the regular expression $(a|ab)(b|)$.

Paper A2 Example:. $r_2 = (a + b + ab)^*$ against $w = ab$,

8.2 Dataset Preparation

Two kinds of input drive the measurements in this chapter, and they are kept deliberately distinct rather than pooled into one corpus.

8.2.1 Synthetic Patterns

The synthetic patterns used throughout this thesis, `paper_r1`, `paper_r2`, the pathological $(a^*)^*$ -style expressions of Section ??, and the benchmark families of Section 8.3, are deliberately small and hand-constructed, each chosen to isolate one specific algorithmic property at a time rather than to resemble real regex use:

Family	Pattern shape	Property isolated
<code>small_patterns</code>	<code>a</code> , <code>a*</code> against <code>""/"/"a"/"aaa"</code>	fixed per-call overhead
<code>scaling_a_star</code>	<code>a*</code> against <code>aⁿ</code>	cost as a function of input length alone
<code>deep_expression</code>	<code>a · a ··· a</code> , depth n	cost as a function of expression size
<code>ambiguous_star</code>	<code>(a + aa)*</code> against <code>aⁿ</code>	branching from genuine ambiguity
<code>pathological, benign</code>	$(a^*)^*$ vs. $(a + b)^*$	the naive matcher's worst and ordinary case
<code>nesting_depth</code>	<code>a*</code> nested n times	cost as a function of nesting, not length

They say little on their own about how any of the four parsers behaves on the regular expressions people actually write, which is what Section 8.2.2 closes the gap on; kept separate, each still isolates one variable the mixed real-world corpus cannot.

8.2.2 The Mamouras Regex Dataset

This thesis's own real-world corpus was independently assembled from three families of real-world regular expression sources chosen to cover different "shapes" of real regex use, not three samples of the same thing:

- **Snort/Suricata** network intrusion-detection signatures (`pcre:"..."` rule options, sourced from the Emerging Threats Open ruleset), literal-heavy, moderate complexity, the closest of the three to "everyday" regex use.
- **SpamAssassin** email spam-filtering rules (`body/header` Perl-regex rules), content matching over unstructured text, a different shape than network signatures.
- **RegexLib** crowd-sourced general-purpose patterns (validators, extractors), the opposite end, where chained lookahead assertions are idiomatic, stress-testing the frontend's rejection path hardest of the three.

Extraction.

Filtering.

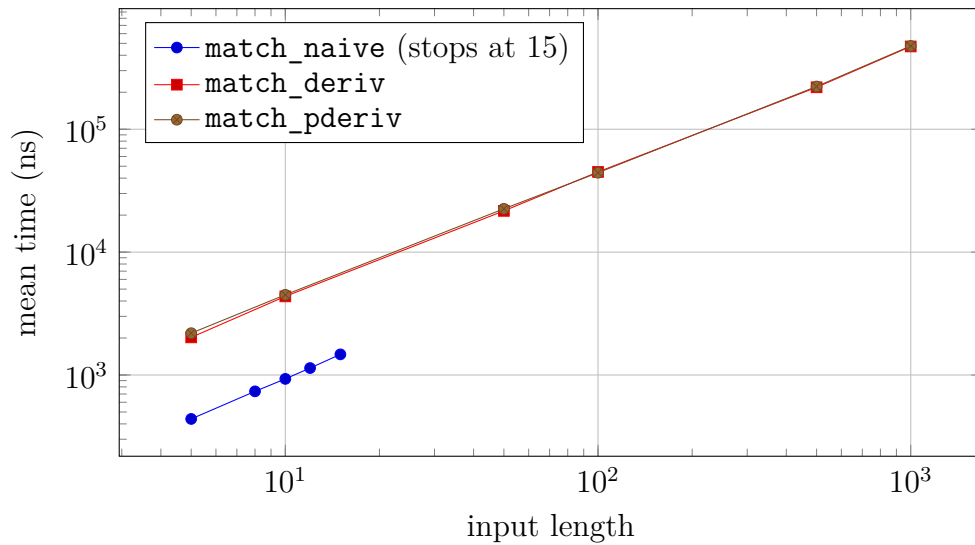
Format	Extracted	Accepted	Backreference	Lookaround	Other
Suricata	231	209 (90.5%)	5	11	6
SpamAssassin	82	69 (84.1%)	1	11	1
RegexLib	499	407 (81.6%)	8	55	29
Total	812	685 (84.4%)	14	77	36

Generating Input.

8.3 Performance Analysis

8.3.1 Performance on Synthetic Patterns

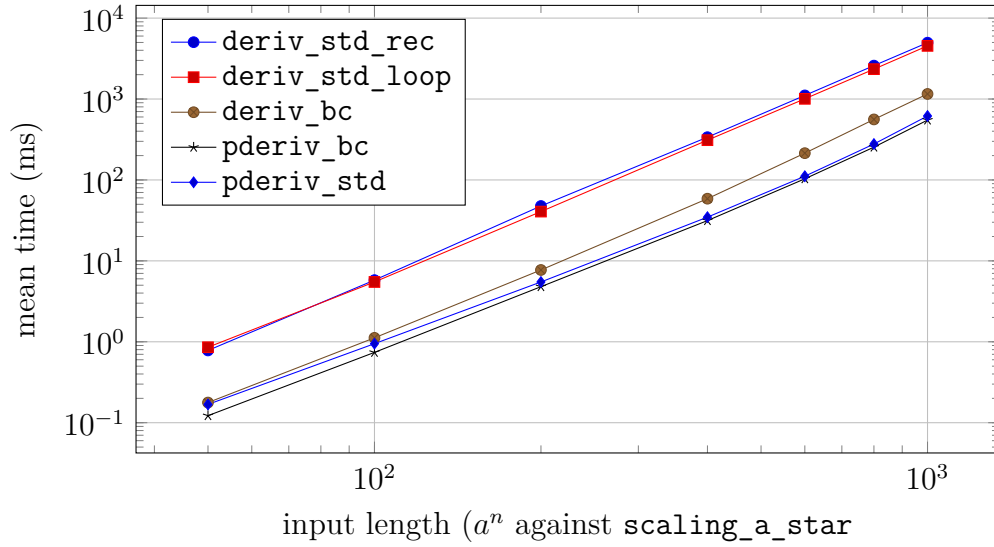
Comparing The Three Matchers.



Stack depth: recursive vs. iterative construction. Comparing recursive and loop-based standard derivative parser.

Heap allocation: deriv_std vs. deriv_bc. Savings without backward pass.

Derivative-based vs. partial-derivative-based.



Benchmark results.

Parser	$n=5$	$n=10$	$n=15$	$n=18$
deriv_std_rec	8.0 μ s	110.7 μ s	1.427 ms	6.052 ms
deriv_bc	11.4 μ s	183.8 μ s	4.175 ms	49.491 ms
pderiv_bc	13.4 μ s	210.6 μ s	2.207 ms	9.236 ms
pderiv_std	22.4 μ s	323.9 μ s	3.374 ms	14.263 ms

8.3.2 Evaluation on the Mamouras Regex Dataset

- Corpus sample sweep
- Neutral-case probe
- Worst-case probe
- Best-case probe

8.4 Discussion of Results

8.4.1 Optimization Challenges and Insights

Character classes in place of literal alternation.

Frontier-wide deduplication.

9 Related Work and Conclusion

9.1 Related Work

9.2 Conclusion

Bibliography

- Valentin Antimirov. Partial derivatives of regular expressions and finite automaton constructions. *Theoretical Computer Science*, 155(2):291–319, 1996. doi: 10.1016/0304-3975(95)00182-4.
- Janusz A. Brzozowski. Derivatives of regular expressions. *Journal of the ACM*, 11(4): 481–494, 1964. doi: 10.1145/321239.321249.
- Russ Cox. Regular expression matching can be simple and fast (but is slow in Java, Perl, PHP, Python, Ruby, ...). <https://swtch.com/~rsc/regexp/regexp1.html>, 2007. Accessed: 2026-08-05.
- Alain Frisch and Luca Cardelli. Greedy regular expression matching. In *Automata, Languages and Programming (ICALP 2004)*, volume 3142, pages 618–629. Springer, 2004. doi: 10.1007/978-3-540-27836-8_53.
- IEEE and The Open Group. Posix.1-2017: Base definitions, chapter 9 (regular expressions), 2017. IEEE Std 1003.1-2017.
- Konstantinos Mamouras, Alexis Le Glaunec, Wu Angela Li, and Agnishom Chattopadhyay. Static analysis for checking the disambiguation robustness of regular expressions. *Proceedings of the ACM on Programming Languages*, 8(PLDI):231, 2024. doi: 10.1145/3656461.
- Martin Sulzmann. Programming paradigms: Regular expressions. <https://sulzmann.github.io/ProgrammingParadigms/pp-regular-expressions.html>, 2026. Accessed: 2026-08-08.
- Martin Sulzmann and Kenny Zhuo Ming Lu. Regular expression sub-matching using partial derivatives. In *PPDP*, pages 79–90, 2012. doi: 10.1145/2370776.2370788.
- Martin Sulzmann and Kenny Zhuo Ming Lu. Posix regular expression parsing with derivatives. In *Functional and Logic Programming (FLOPS 2014)*, volume 8475. Springer, 2014.
- The Rust Project Developers. Using `Box<T>` to point to data on the heap. The Rust Programming Language, 2026a. URL <https://doc.rust-lang.org/book/ch15-01-box.html>. Accessed: 2026-09-11.
- The Rust Project Developers. Types. The Rust Reference, 2026b. URL <https://doc.rust-lang.org/reference/types.html>. Accessed: 2026-09-11.

A Diagnostics

```
cargo run -- parse "(a|ab)(b|)" <INPUT> --parser <PARSER> --diag 3
```

Field	Value
Expression	"(a ab)(b)", that is $(a + ab)(b + \varepsilon)$
Inputs	"ab" (match), "b" (no match)
Parsers	deriv_std_rec, deriv_bc, pderiv_std, pderiv_bc
Output file	reports/report.txt
Omitted	the Timestamp line
Wrapped	long lines
∂	d
.	.
$\oplus$	(+)
ε	epsilon
ϕ	phi
✓	[OK]
Policy	the implementation's internal label, not a claim about disambiguation policy

A.1 deriv_std_rec, match

```
$ cargo run -- parse "(a|ab)(b|)" "ab" \
  --parser deriv_std_rec --diag 3
```

```
=====
REGEX ENGINE DEBUG REPORT
```

```
=====
Mode:                POSIX Standard (recursive parser)
Regex:               (a|ab)(b|)
Input:               "ab"
Result:              MATCH
Policy:              POSIX
```

```
=====
TIMING
```

```
=====
Parse time:          0.012ms
Expressions stored:  3 (r0 -> r1 -> r2)
```

```
=====
FORWARD PASS (Derivatives)
```

```
r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))
```

```
Step 1: d(r0, 'a')
```

```
rule: d(r1.r2, 'a') [nullable(r1)=false]
```

```
= d(r1, 'a').r2
```

```
r1 = Seq(Alt(Eps, Seq(Eps, Lit('b'))), Alt(Lit('b'), Eps))
```

```
nullable(r1) = true [OK] position 1 matched
```

```
Step 2: d(r1, 'b')
```

```
rule: d(r1.r2, 'b') [nullable(r1)=true]
```

```
= d(r1, 'b').r2 + d(r2, 'b')
```

```
r2 = Alt(Seq(Alt(Phi, Alt(Seq(Phi, Lit('b'))), Eps)), Alt(Lit('b'), Eps)), Alt(Eps, Phi))
```

```
nullable(r2) = true [OK] position 2 matched
```

```
-----=====
```

```
NULLABILITY CHECK
```

```
-----=====
```

```
nullable(r2) = true -> proceed to backward pass
```

```
-----=====
```

```
BACKWARD PASS (mkEps + inject)
```

```
-----=====
```

```
mkEps(r2):
```

```
mkEps(r1 + r2): nullable(r1)=true -> Left(mkEps(r1))
```

```
v2 = Left (Right Right (), Right ())
```

```
inject(r1, 'b', v2):
```

```
inject(Seq(Alt(Eps, Seq(Eps, Lit('b'))), Alt(Lit('b'), Eps)), 'b',  
Left (Right Right (), Right ()))
```

```
-> (Right ((), b), Right ())
```

```
v1 = (Right ((), b), Right ())
```

```
inject(r0, 'a', v1):
```

```
inject(Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'),  
Eps)), 'a', (Right ((), b), Right ()))
```

```
-> (Right (a, b), Right ())
```

```
v0 = (Right (a, b), Right ())
```

```
-----=====
```

```
RESULT
```

```
-----=====
```

```
Parse tree: (Right (a, b), Right ())
```

```
Flattened: "ab"
```

```
-----=====
```

```
END OF REPORT
```

```
-----=====
```

A.2 deriv_bc, match

```
$ cargo run -- parse "(a|ab)(b|)" "ab" \
  --parser deriv_bc --diag 3
-----=====
REGEX ENGINE DEBUG REPORT
-----=====
Mode:                POSIX Bitcoded
Regex:               (a|ab)(b|)
Input:               "ab"
Result:              MATCH
Policy:              POSIX
-----=====
TIMING
-----=====
Parse time:          0.015ms
Derivative steps:    2
-----=====
INTERNALIZE
-----=====
internalize((a|ab)(b|))
  rule: internalize(r1 . r2) = []@(internalize r1)(internalize r2)
ri0 = []@([]@([]@([false]@'a' (+) [true]@([]@'a' . []@'b')) .
  []@([false]@'b' (+) [true]@epsilon))
-----=====
FORWARD PASS (deriv_bc + simp)
-----=====
Step 1 (char: 'a', position 1)
ri0 = []@([]@([]@([false]@'a' (+) [true]@([]@'a' . []@'b')) .
  []@([false]@'b' (+) [true]@epsilon))
deriv_bc(ri0, 'a'):
  rule: (bs@ri1.ri2) \ 'a' [nullable_bc(ri1)=false]
    = bs@((ri1\'a'.ri2)
simp -> []@([]@([false]@epsilon (+) [true]@'b') . []@([false]@'b' (+)
  [true]@epsilon))
ri1 = []@([]@([false]@epsilon (+) [true]@'b') . []@([false]@'b' (+)
  [true]@epsilon))
  nullable_bc(ri1) = true [OK] bits accumulating

Step 2 (char: 'b', position 2)
ri1 = []@([]@([false]@epsilon (+) [true]@'b') . []@([false]@'b' (+)
  [true]@epsilon))
deriv_bc(ri1, 'b'):
  rule: (bs@ri1.ri2) \ 'b' [nullable_bc(ri1)=true]
    = bs@((ri1\'b'.ri2) (+) (fuse (mkEpsBC ri1) (ri2\'b')))
simp -> []@([true]@([false]@'b' (+) [true]@epsilon) (+) [false,
  false]@epsilon)
ri2 = []@([true]@([false]@'b' (+) [true]@epsilon) (+) [false,
```

```

    false]@epsilon)
nullable_bc(ri2) = true   [OK]   bits accumulating

-----=====
NULLABILITY CHECK
-----=====
nullable_bc(ri2) = true -> proceed to mkEpsBC + decode
-----=====
MKEEPSBC + DECODE
-----=====
mkEpsBC(ri2) = [11]

decode((a|ab)(b|), [11]):
  = (Right (a, b), Right ())
-----=====
RESULT
-----=====
Bits:                [11]
Parse tree:          (Right (a, b), Right ())
Flattened:           "ab"
-----=====
END OF REPORT
-----=====

```

A.3 pderiv_std, match

```

$ cargo run -- parse "(a|ab)(b|)" "ab" \
  --parser pderiv_std --diag 3
-----=====
REGEX ENGINE DEBUG REPORT
-----=====
Mode:                Standard Partial-Derivative (pderiv_std)
Regex:               (a|ab)(b|)
Input:               "ab"
Result:              MATCH
Policy:              GREEDY
-----=====
TIMING
-----=====
Parse time:          0.017ms
Steps:               2
-----=====
INITIAL FRONTIER
-----=====
[r0]  where r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))),
  Alt(Lit('b'), Eps))
-----=====
FORWARD PASS (pderiv_tree per residual, frontier flattened)

```

```

-----=====
Step 1 (char: 'a', position 1)
  frontier before (1 residual(s)):
[0] Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))
  pderiv_tree('a', .) applied to each:
    from [0]:
      rule: pderiv_tree('a', r1.r2) [nullable(r1)=false]
= [(r1'.r2, inj: continue r1) | (r1',_) <- pderiv_tree(c,r1)]
-> Seq(Eps, Alt(Lit('b'), Eps)) (+ injection closure, not printable)
-> Seq(Seq(Eps, Lit('b')), Alt(Lit('b'), Eps)) (+ injection closure,
      not printable)
  frontier after (2 residual(s)):
    [0] Seq(Eps, Alt(Lit('b'), Eps))
    [1] Seq(Seq(Eps, Lit('b')), Alt(Lit('b'), Eps))
  a residual in this frontier is nullable [OK] parse tree available

Step 2 (char: 'b', position 2)
  frontier before (2 residual(s)):
    [0] Seq(Eps, Alt(Lit('b'), Eps))
    [1] Seq(Seq(Eps, Lit('b')), Alt(Lit('b'), Eps))
  pderiv_tree('b', .) applied to each:
    from [0]:
      rule: pderiv_tree('b', r1.r2) [nullable(r1)=true]
= nub_tree $ [(r1'.r2, inj: continue r1) | (r1',_) <-
      pderiv_tree(c,r1)]
++ [(r2', inj: mk_eps(r1) paired with r2's result) | (r2',_) <-
      pderiv_tree(c,r2)]
    -> Eps (+ injection closure, not printable)
    from [1]:
      rule: pderiv_tree('b', r1.r2) [nullable(r1)=false]
= [(r1'.r2, inj: continue r1) | (r1',_) <- pderiv_tree(c,r1)]
-> Seq(Eps, Alt(Lit('b'), Eps)) (+ injection closure, not printable)
  frontier after (2 residual(s)):
    [0] Eps
    [1] Seq(Eps, Alt(Lit('b'), Eps))
  a residual in this frontier is nullable [OK] parse tree available

-----=====
SELECTION (first nullable residual, priority/list order)
-----=====
any nullable residual in final frontier = true -> proceed to injection
(already applied by select)
-----=====
RESULT
-----=====
Parse tree:          (Left a, Left b)
Flattened:           "ab"
-----=====

```

END OF REPORT

A.4 pderiv_bc, match

```
$ cargo run -- parse "(a|ab)(b|)" "ab" \
  --parser pderiv_bc --diag 3
```

REGEX ENGINE DEBUG REPORT

```
Mode:                Bitcoded Partial-Derivative (pDerivBC)
Regex:               (a|ab)(b|)
Input:               "ab"
Result:              MATCH
Policy:              GREEDY
```

TIMING

```
Parse time:          0.012ms
Steps:               2
```

INITIAL FRONTIER

```
[(r0, [])]  where r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))),
  Alt(Lit('b'), Eps))
```

FORWARD PASS (pDerivBC per residual, frontier flattened)

Step 1 (char: 'a', position 1)

frontier before (1 residual(s)):

```
[0] Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))
  bits-so-far = []
```

pDerivBC('a', .) applied to each:

from [0]:

```
rule: pDerivBC('a', r1.r2) [nullable(r1)=false]
      = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
-> (Alt(Lit('b'), Eps), [0])
-> (Seq(Lit('b'), Alt(Lit('b'), Eps)), [1])
```

frontier after (2 residual(s)):

```
[0] Alt(Lit('b'), Eps)  bits = [0]
[1] Seq(Lit('b'), Alt(Lit('b'), Eps))  bits = [1]
```

a residual in this frontier is nullable [OK] bits accumulating

Step 2 (char: 'b', position 2)

frontier before (2 residual(s)):

```
[0] Alt(Lit('b'), Eps)  bits-so-far = [0]
[1] Seq(Lit('b'), Alt(Lit('b'), Eps))  bits-so-far = [1]
```

pDerivBC('b', .) applied to each:

```

    from [0]:
rule: pDerivBC('b', r1+r2) = tag(0, pDerivBC('b',r1)) ++ tag(1,
    pDerivBC('b',r2)), nub2
    -> (Eps, [0])
    from [1]:
        rule: pDerivBC('b', r1.r2) [nullable(r1)=false]
            = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
            -> (Alt(Lit('b'), Eps), [])
frontier after (2 residual(s)):
    [0] Eps    bits = [00]
    [1] Alt(Lit('b'), Eps)    bits = [1]
a residual in this frontier is nullable [OK] bits accumulating

=====
SELECTION (first nullable residual, priority/list order)
=====
any nullable residual in final frontier = true -> proceed to mkEpsBC +
    decode
=====
MKEEPSBC + DECODE
=====
selected residual's accumulated bits ++ mkEpsBC(residual) = [00]

decode((a|ab)(b|), [00]):
    = (Left a, Left b)
=====
RESULT
=====
Bits:                [00]
Parse tree:          (Left a, Left b)
Flattened:           "ab"
=====
END OF REPORT
=====

```

A.5 deriv_std_rec, no match

```

$ cargo run -- parse "(a|ab)(b|)" "b" \
    --parser deriv_std_rec --diag 3
=====
REGEX ENGINE DEBUG REPORT
=====
Mode:                POSIX Standard (recursive parser)
Regex:              (a|ab)(b|)
Input:              "b"
Result:             NO MATCH
Policy:             POSIX
=====

```

```

TIMING
=====
Parse time:          0.005ms
Expressions stored:  2 (r0 -> r1)
=====
FORWARD PASS (Derivatives)
=====
r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))

Step 1: d(r0, 'b')
  rule: d(r1.r2, 'b') [nullable(r1)=false]
        = d(r1, 'b').r2
r1 = Seq(Alt(Phi, Seq(Phi, Lit('b'))), Alt(Lit('b'), Eps))
  nullable(r1) = false [FAIL] position 1

=====
NULLABILITY CHECK
=====
nullable(r1) = false -> no parse tree exists for full input
=====
PARTIAL RECOVERY
=====
No prefix matched.
=====
ERROR SUMMARY
=====
Failure at position:  1 (1-indexed)
Found:                'b'
Expected:              'a'
  b
  ^
=====
END OF REPORT
=====

```

A.6 deriv_bc, no match

```

$ cargo run -- parse "(a|ab)(b|)" "b" \
  --parser deriv_bc --diag 3

```

```

=====
REGEX ENGINE DEBUG REPORT
=====
Mode:                POSIX Bitcoded
Regex:               (a|ab)(b|)
Input:               "b"
Result:              NO MATCH
Policy:              POSIX
=====

```

```

TIMING
=====
Parse time:          0.008ms
Derivative steps:    1
=====
INTERNALIZE
=====
internalize((a|ab)(b|))
  rule: internalize(r1 . r2) = []@(internalize r1)(internalize r2)
ri0 = []@([]@([]@([false]@'a' (+) [true]@([]@'a' . []@'b')) .
  []@([false]@'b' (+) [true]@epsilon))
=====
FORWARD PASS (deriv_bc + simp)
=====
Step 1 (char: 'b', position 1)
ri0 = []@([]@([]@([false]@'a' (+) [true]@([]@'a' . []@'b')) .
  []@([false]@'b' (+) [true]@epsilon))
deriv_bc(ri0, 'b'):
  rule: (bs@ri1.ri2) \ 'b' [nullable_bc(ri1)=false]
    = bs@(ri1\'b'.ri2)
simp -> phi
ri1 = phi
nullable_bc(ri1) = false [FAIL]
=====
NULLABILITY CHECK
=====
nullable_bc(ri1) = false -> no full parse tree exists
=====
PARTIAL RECOVERY
=====
No prefix matched.
=====
ERROR SUMMARY
=====
Failure at position:  1 (1-indexed)
Found:                'b'
Expected:              'a'
  b
  ^
=====
END OF REPORT
=====

```

A.7 pderiv_std, no match

```

$ cargo run -- parse "(a|ab)(b|)" "b" \
  --parser pderiv_std --diag 3

```

```

=====
REGEX ENGINE DEBUG REPORT
=====
Mode:                Standard Partial-Derivative (pderiv_std)
Regex:               (a|ab)(b|)
Input:               "b"
Result:              NO MATCH
Policy:              GREEDY
=====

TIMING
=====
Parse time:          0.007ms
Steps:               1
=====

INITIAL FRONTIER
=====
[r0]  where r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))),
    Alt(Lit('b'), Eps))
=====

FORWARD PASS (pderiv_tree per residual, frontier flattened)
=====
Step 1 (char: 'b', position 1)
  frontier before (1 residual(s)):
[0] Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))
  pderiv_tree('b', .) applied to each:
    from [0]:
      rule: pderiv_tree('b', r1.r2) [nullable(r1)=false]
= [(r1'.r2, inj: continue r1) | (r1',_) <- pderiv_tree(c,r1)]
  -> []
  frontier after (0 residual(s)):
  no nullable residual in this frontier [FAIL]

=====

SELECTION (first nullable residual, priority/list order)
=====
any nullable residual in final frontier = false -> no full parse tree
exists
=====

PARTIAL RECOVERY
=====
No prefix matched.
=====

ERROR SUMMARY
=====
Failure at position: 1 (1-indexed)
Found:               'b'
Expected:             'a'
  b

```

```

~
=====
END OF REPORT
=====

```

A.8 pderiv_bc, no match

```

$ cargo run -- parse "(a|ab)(b|)" "b" \
  --parser pderiv_bc --diag 3
=====

```

REGEX ENGINE DEBUG REPORT

```

=====
Mode:                Bitcoded Partial-Derivative (pDerivBC)
Regex:               (a|ab)(b|)
Input:               "b"
Result:              NO MATCH
Policy:              GREEDY
=====

```

TIMING

```

=====
Parse time:          0.006ms
Steps:               1
=====

```

INITIAL FRONTIER

```

=====
[(r0, [])]  where r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))),
  Alt(Lit('b'), Eps))
=====

```

FORWARD PASS (pDerivBC per residual, frontier flattened)

```

=====
Step 1 (char: 'b', position 1)
  frontier before (1 residual(s)):
[0] Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))
    bits-so-far = []
  pDerivBC('b', .) applied to each:
    from [0]:
      rule: pDerivBC('b', r1.r2) [nullable(r1)=false]
          = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
      -> []
  frontier after (0 residual(s)):
  no nullable residual in this frontier [FAIL]
=====

```

SELECTION (first nullable residual, priority/list order)

```

=====
any nullable residual in final frontier = false -> no full parse tree
exists
=====

```

PARTIAL RECOVERY

No prefix matched.

ERROR SUMMARY

Failure at position: 1 (1-indexed)
Found: 'b'
Expected: 'a'
b
^

END OF REPORT